Student Dropout Predictor

Objective

The objective of this analysis is to develop a machine learning model capable of predicting whether a university student will drop out or successfully graduate based on demographic and academic indicators available early in the academic journey. Multiple machine learning algorithms will be evaluated and compared in order to identify the model that provides the most reliable predictions for early intervention systems.

The analysis follows a structured data science workflow consisting of:

1. Data Understanding
2. Exploratory Data Analysis
3. Data Preparation
4. Model Training
5. Model Evaluation

1. Import Libraries

In [1]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc
from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import cross_val_score

sns.set(style="whitegrid")
```

2. Load Dataset

The dataset used in this project contains demographic, academic, and socio-economic indicators describing university students. The goal is to predict whether a student will drop out or successfully graduate.

In [2]:
```
df = pd.read_csv("../datasets/data.csv", sep=";")

# Clean column names
df.columns = df.columns.str.replace("\t", "")
df.columns = df.columns.str.strip()
```

```
df.columns = df.columns.str.replace("'", "")

df.head()
```

Out[2]:

	Marital status	Application mode	Application order	Course	Daytime/evening attendance	Previous qualification	Previo qualificati (grad
0	1	17	5	171	1	1	12
1	1	15	1	9254	1	1	16
2	1	1	5	9070	1	1	12
3	1	17	2	9773	1	1	12
4	2	39	1	8014	0	1	10

5 rows × 37 columns

3. Dataset Overview

Before building machine learning models, it is important to understand the structure of the dataset.

Shape

In [3]:
```
df.shape
```

Out[3]: (4424, 37)

Column Names

In [4]:
```
df.columns
```

Out[4]:
```
Index(['Marital status', 'Application mode', 'Application order', 'Course',
       'Daytime/evening attendance', 'Previous qualification',
       'Previous qualification (grade)', 'Nacionality',
       'Mothers qualification', 'Fathers qualification', 'Mothers occupation',
       'Fathers occupation', 'Admission grade', 'Displaced',
       'Educational special needs', 'Debtor', 'Tuition fees up to date',
       'Gender', 'Scholarship holder', 'Age at enrollment', 'International',
       'Curricular units 1st sem (credited)',
       'Curricular units 1st sem (enrolled)',
       'Curricular units 1st sem (evaluations)',
       'Curricular units 1st sem (approved)',
       'Curricular units 1st sem (grade)',
       'Curricular units 1st sem (without evaluations)',
       'Curricular units 2nd sem (credited)',
       'Curricular units 2nd sem (enrolled)',
       'Curricular units 2nd sem (evaluations)',
       'Curricular units 2nd sem (approved)',
       'Curricular units 2nd sem (grade)',
       'Curricular units 2nd sem (without evaluations)', 'Unemployment rate',
       'Inflation rate', 'GDP', 'Target'],
      dtype='str')
```

Feature Description

The dataset contains a large number of demographic, academic, and socioeconomic features. Describing every feature individually would be impractical, therefore selected key features are highlighted below to provide an overview of the most relevant variables used in the analysis.

Feature	Description
Admission grade	Student’s grade at the time of admission
Age at enrollment	Age of the student when enrolling
Debtor	Indicates whether the student has unpaid tuition fees
Tuition fees up to date	Whether the student has paid tuition fees on time
Curricular units 1st sem (approved)	Number of courses successfully completed in first semester
Curricular units 1st sem (grade)	Average grade in first semester
Scholarship holder	Whether the student receives a scholarship
Application mode	Method used to apply to the university

The features can be broadly grouped into the following categories:

- **Demographic features**: Gender, nationality, age
- **Academic performance features**: Grades, approved units, evaluations
- **Administrative/financial features**: Tuition status, debtor status, scholarship
- **Macroeconomic indicators**: GDP, inflation rate, unemployment rate

Data Types

In [5]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4424 entries, 0 to 4423
Data columns (total 37 columns):
 #   Column                                          Non-Null Count  Dtype
---  ------                                          --------------  -----
 0   Marital status                                  4424 non-null   int64
 1   Application mode                                4424 non-null   int64
 2   Application order                               4424 non-null   int64
 3   Course                                          4424 non-null   int64
 4   Daytime/evening attendance                      4424 non-null   int64
 5   Previous qualification                          4424 non-null   int64
 6   Previous qualification (grade)                  4424 non-null   float64
 7   Nacionality                                     4424 non-null   int64
 8   Mothers qualification                           4424 non-null   int64
 9   Fathers qualification                           4424 non-null   int64
 10  Mothers occupation                              4424 non-null   int64
 11  Fathers occupation                              4424 non-null   int64
 12  Admission grade                                 4424 non-null   float64
 13  Displaced                                       4424 non-null   int64
 14  Educational special needs                       4424 non-null   int64
 15  Debtor                                          4424 non-null   int64
 16  Tuition fees up to date                         4424 non-null   int64
 17  Gender                                          4424 non-null   int64
 18  Scholarship holder                              4424 non-null   int64
 19  Age at enrollment                               4424 non-null   int64
 20  International                                   4424 non-null   int64
 21  Curricular units 1st sem (credited)             4424 non-null   int64
 22  Curricular units 1st sem (enrolled)             4424 non-null   int64
 23  Curricular units 1st sem (evaluations)          4424 non-null   int64
 24  Curricular units 1st sem (approved)             4424 non-null   int64
 25  Curricular units 1st sem (grade)                4424 non-null   float64
 26  Curricular units 1st sem (without evaluations)  4424 non-null   int64
 27  Curricular units 2nd sem (credited)             4424 non-null   int64
 28  Curricular units 2nd sem (enrolled)             4424 non-null   int64
 29  Curricular units 2nd sem (evaluations)          4424 non-null   int64
 30  Curricular units 2nd sem (approved)             4424 non-null   int64
 31  Curricular units 2nd sem (grade)                4424 non-null   float64
 32  Curricular units 2nd sem (without evaluations)  4424 non-null   int64
 33  Unemployment rate                               4424 non-null   float64
 34  Inflation rate                                  4424 non-null   float64
 35  GDP                                             4424 non-null   float64
 36  Target                                          4424 non-null   str
dtypes: float64(7), int64(29), str(1)
memory usage: 1.2 MB
```

Missing Values Check

In [6]:
```
missing_values = df.isnull().sum()
missing_values[missing_values > 0]
```

Out[6]:
```
Series([], dtype: int64)
```

The dataset was checked for missing values to ensure data quality before further analysis. The results show that there are no missing values in any of the features. Therefore, no imputation or data cleaning related to missing values is required.

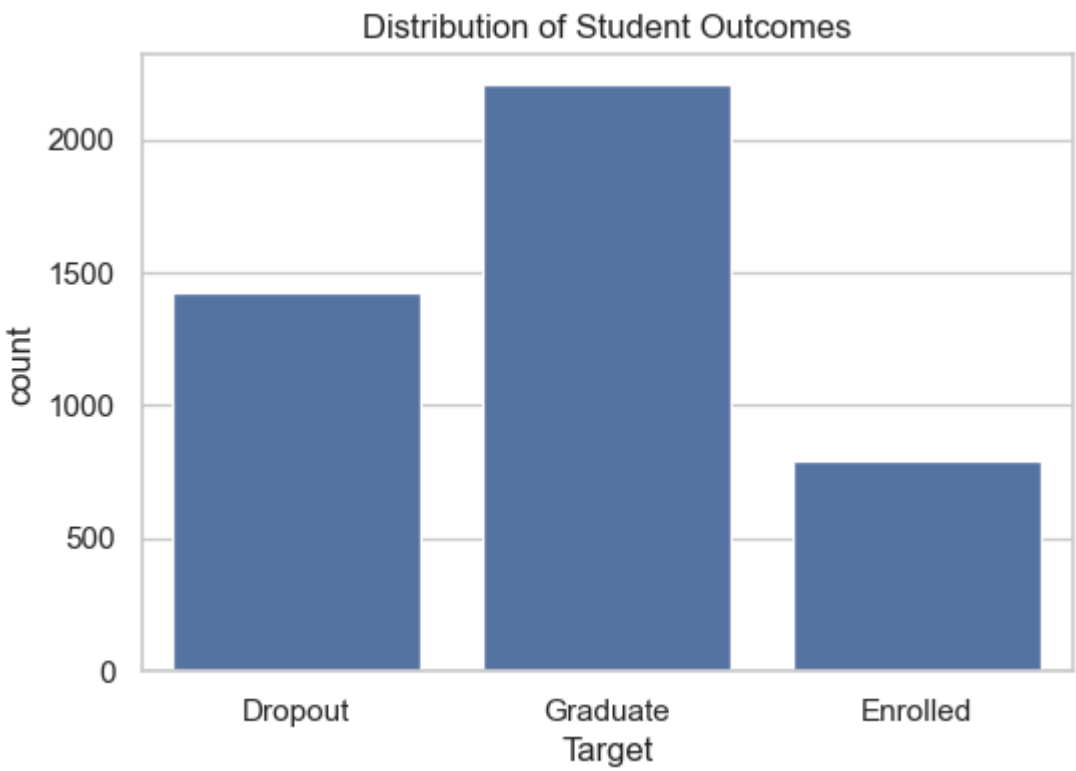
4. Target Variable Distribution

The dataset contains three possible outcomes:

- Dropout
- Enrolled
- Graduate

Since the final outcome of "Enrolled" students is unknown, these observations will later be removed from the dataset.

In [7]:
```
df["Target"].value_counts()

plt.figure(figsize=(6,4))
sns.countplot(x="Target", data=df)
plt.title("Distribution of Student Outcomes")
plt.show()
```



5. Filtering the Dataset

The dataset originally contains three possible academic outcomes: **Dropout**, **Enrolled**, and **Graduate**.

The *Enrolled* category represents students whose final academic outcome is not yet known. These students may eventually either graduate or drop out in the future. Including these observations in the training data would therefore introduce label uncertainty, because the model would be trained on examples with incomplete outcomes.

To ensure that the model learns only from observations with clearly defined final outcomes, records labeled **Enrolled** are removed from the dataset.

The prediction task therefore becomes a `binary classification problem`:

1 = **Dropout**
0 = **Graduate**

In [8]:
```
# Remove whitespace
df["Target"] = df["Target"].str.strip()

df = df[df["Target"] != "Enrolled"]
```

In [9]:
```
df["Target"].value_counts()
```

Out[9]:
```
Target
Graduate    2209
Dropout     1421
Name: count, dtype: int64
```

After removing the "Enrolled" category, the dataset contains 2209 graduate cases and 1421 dropout cases. Although the classes are not perfectly balanced, the distribution remains sufficiently comparable for standard classification metrics such as accuracy, precision, recall, and F1-score to provide meaningful evaluation.

Convert Target to Binary

In [10]:
```
df["Target"] = df["Target"].map({
    "Dropout": 1,
    "Graduate": 0
})

df["Target"].value_counts()
```

Out[10]:
```
Target
0    2209
1    1421
Name: count, dtype: int64
```

In [11]:
```
print(df["Target"].unique())
```

```
[1 0]
```

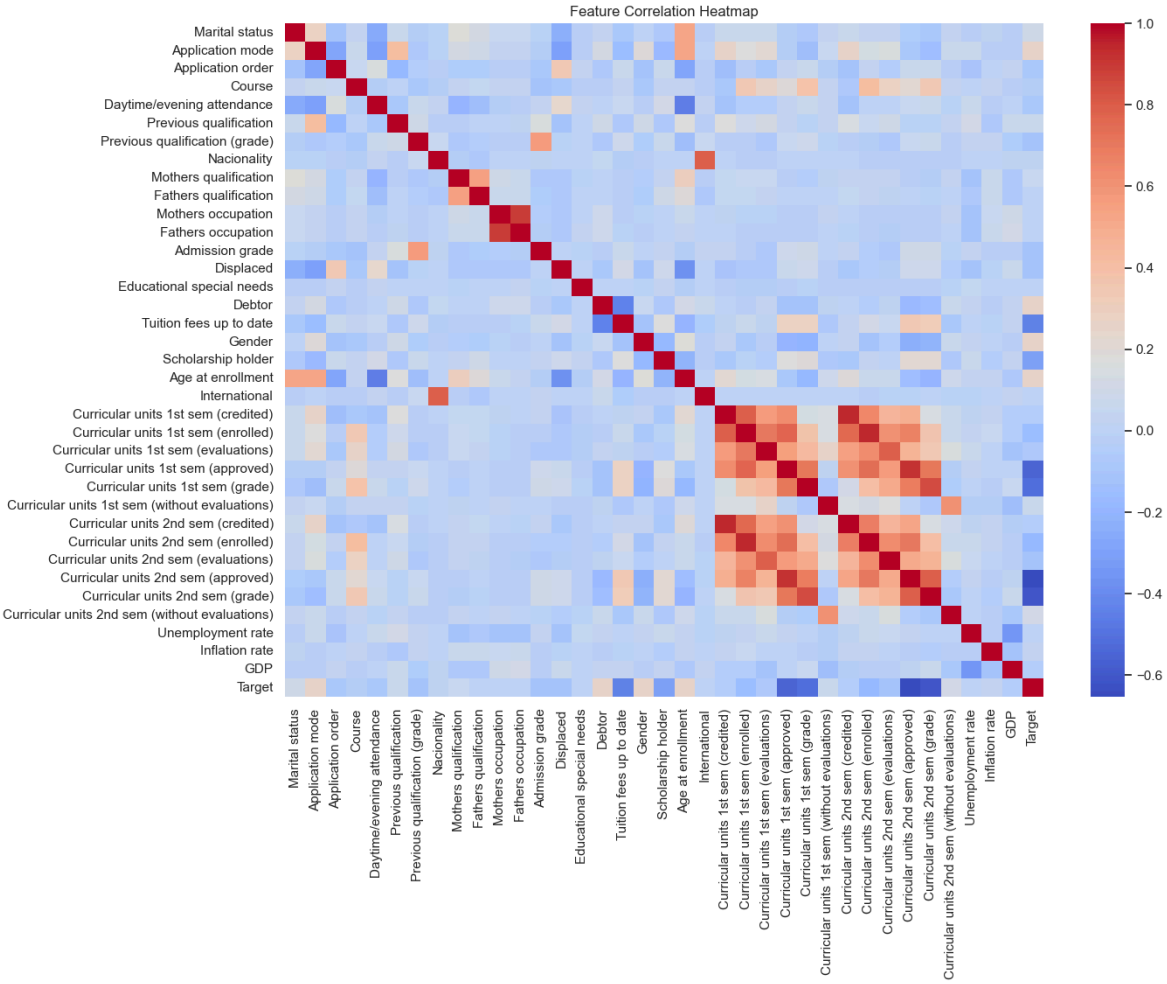
6. Exploratory Data Analysis

Before training machine learning models, exploratory data analysis is conducted to better understand relationships between variables and identify potential predictors of student dropout.

6.1. Correlation Analysis

6.1.1. Correlation Heatmap

In [12]:
```
plt.figure(figsize=(14,10))

sns.heatmap(df.corr(), cmap="coolwarm")

plt.title("Feature Correlation Heatmap")
plt.show()
```



6.1.2. Correlation with Target

In [13]:
```
df.corr()["Target"].drop("Target").sort_values(ascending=False)
```

Out[13]:
```
Age at enrollment                                   0.267229
Debtor                                              0.267207
Gender                                              0.251955
Application mode                                    0.244507
Curricular units 2nd sem (without evaluations)      0.102687
Marital status                                      0.100479
Curricular units 1st sem (without evaluations)      0.074642
Previous qualification                              0.062323
Mothers qualification                               0.053989
Inflation rate                                      0.030326
Nacionality                                         0.015516
Educational special needs                           0.007254
Fathers qualification                               0.005865
Mothers occupation                                 -0.000772
Unemployment rate                                  -0.004198
Fathers occupation                                 -0.005066
International                                      -0.006181
Course                                             -0.038135
Curricular units 1st sem (credited)                -0.046900
GDP                                                -0.050260
Curricular units 2nd sem (credited)                -0.052402
Curricular units 1st sem (evaluations)             -0.059786
Daytime/evening attendance                         -0.084496
Application order                                  -0.094355
Previous qualification (grade)                     -0.109464
Curricular units 2nd sem (evaluations)             -0.119239
Displaced                                          -0.126113
Admission grade                                    -0.128058
Curricular units 1st sem (enrolled)                -0.161074
Curricular units 2nd sem (enrolled)                -0.182897
Scholarship holder                                 -0.313018
Tuition fees up to date                            -0.442138
Curricular units 1st sem (grade)                   -0.519927
Curricular units 1st sem (approved)                -0.554881
Curricular units 2nd sem (grade)                   -0.605350
Curricular units 2nd sem (approved)                -0.653995
Name: Target, dtype: float64
```

6.1.3. Interpretation

The correlation analysis shows that several academic performance indicators have a relatively strong relationship with the target variable. In particular, variables such as the `number of approved curricular units` and `course grades` show the strongest **negative** correlations with student dropout.

The negative correlations indicate that students who successfully complete more courses and achieve higher grades are less likely to drop out of their academic program. This aligns with expectations, as academic performance is often an important factor in student persistence.

Other variables, including `demographic characteristics` and `macroeconomic indicators`, show relatively **weak** correlations with the target variable. This suggests that these features may have a smaller individual influence on student dropout.

It is also important to note that several second-semester academic indicators show strong correlations with the target variable. However, since the objective of this project is

to predict dropout risk early in a student's academic journey, features related to the second semester may introduce `data leakage` and will be considered carefully during the feature selection stage.

Although second-semester variables show strong correlations with the target variable, they are excluded from model training because they would introduce data leakage. These variables would not be available when making early predictions after the first semester.

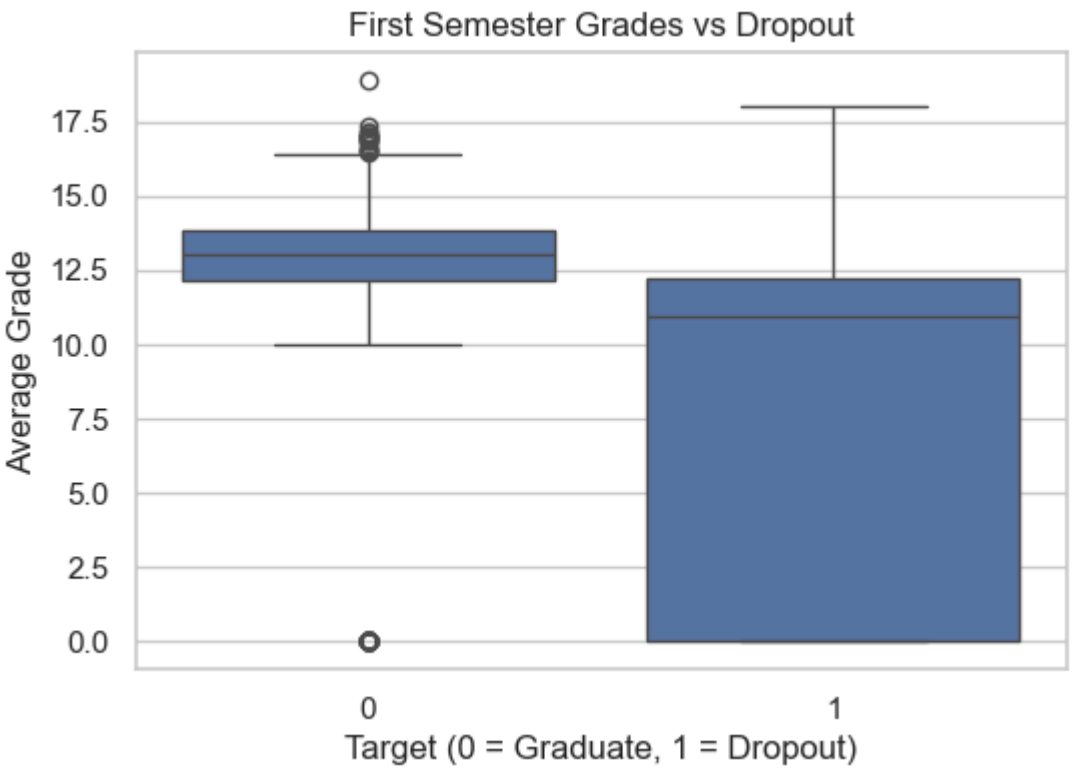
6.2. Feature Relationship Analysis

While correlation analysis provides a general overview of relationships between variables, it does not fully capture how individual features behave across different outcome groups.

To gain deeper insights, selected key variables are analyzed visually to better understand how they differ between students who graduate and those who drop out.

6.2.1. Academic Performance vs Dropout

In [14]:
```
plt.figure(figsize=(6,4))
sns.boxplot(x="Target", y="Curricular units 1st sem (grade)", data=df)

plt.title("First Semester Grades vs Dropout")
plt.xlabel("Target (0 = Graduate, 1 = Dropout)")
plt.ylabel("Average Grade")

plt.show()
```



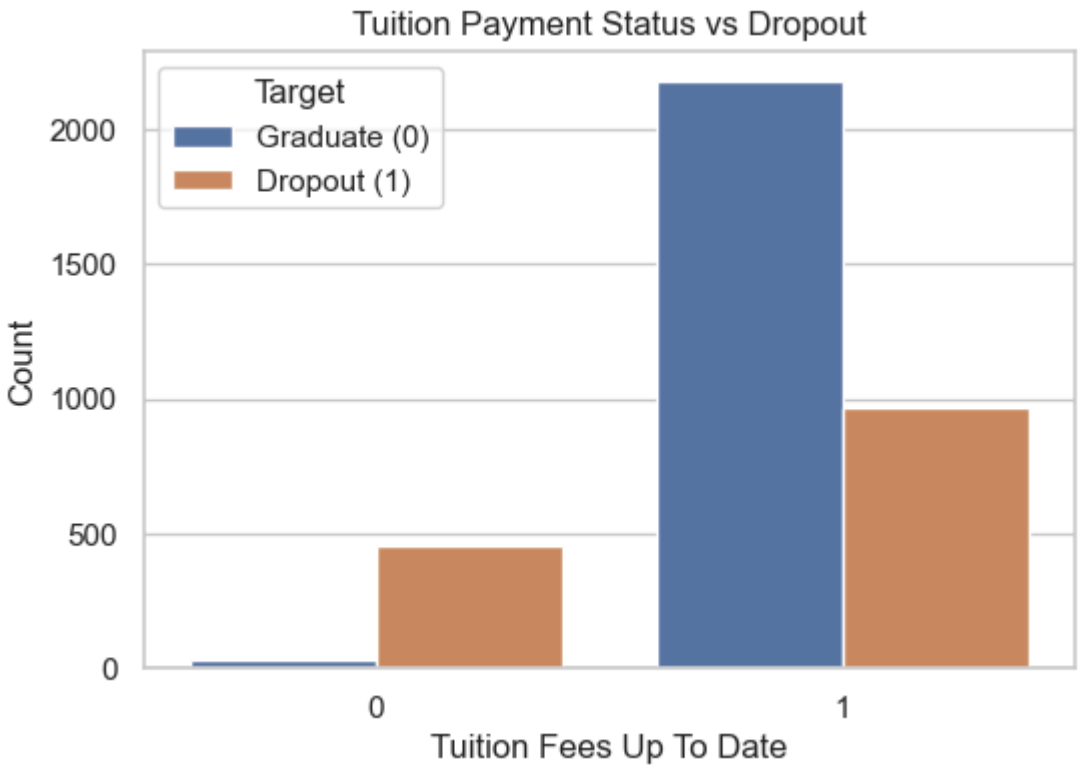
The boxplot reveals a clear separation between students who graduate and those who drop out based on first-semester grades.

Students who graduate (Target = 0) generally achieve higher grades, with a relatively tight distribution centered around higher values. In contrast, students who drop out (Target = 1) show significantly lower median grades and a much wider spread, including a large number of very low or zero grades.

This suggests that poor academic performance early in the program is a strong indicator of future dropout. The presence of many low-grade outliers among dropout students further reinforces the importance of early academic success as a key predictive factor.

6.2.2. Tuition Status vs Dropout

In [15]:
```
plt.figure(figsize=(6,4))
sns.countplot(x="Tuition fees up to date", hue="Target", data=df)

plt.title("Tuition Payment Status vs Dropout")
plt.xlabel("Tuition Fees Up To Date")
plt.ylabel("Count")

plt.legend(title="Target", labels=["Graduate (0)", "Dropout (1)"])

plt.show()
```



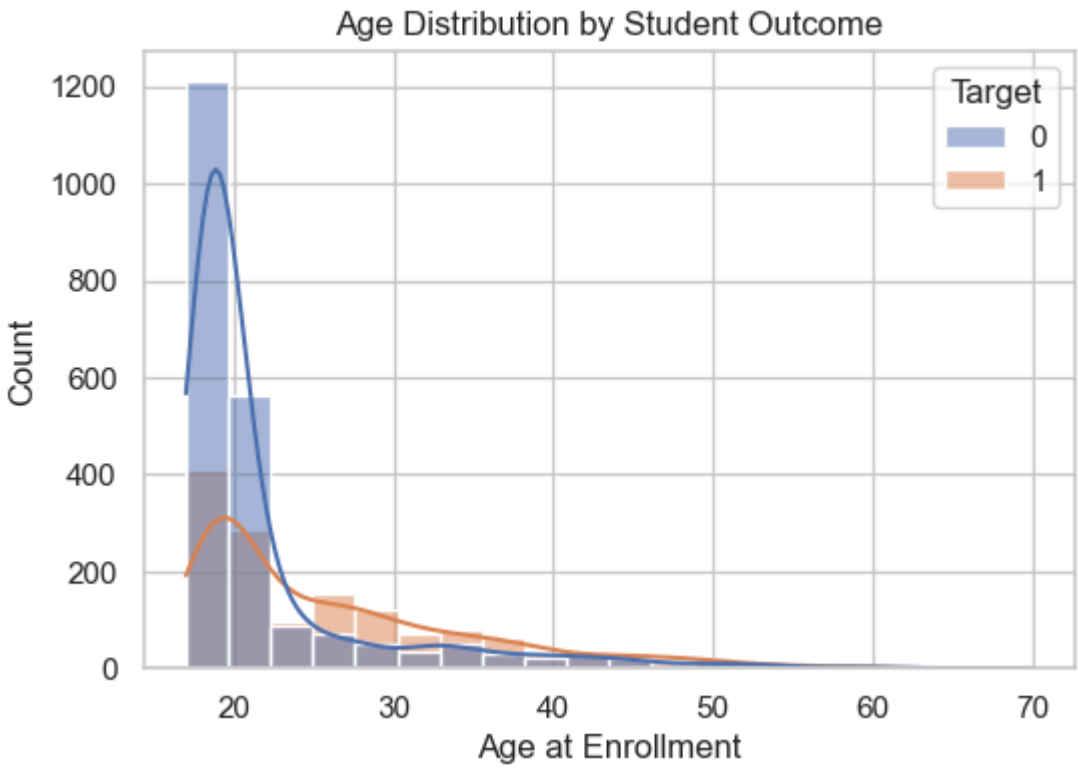
The distribution of tuition payment status shows a strong relationship with student outcomes.

Students who are not up to date with their tuition fees (value = 0) have a noticeably higher proportion of dropout cases compared to graduates. In contrast, students who have their tuition fees up to date (value = 1) are more likely to graduate, although some dropout cases still exist within this group.

This pattern indicates that financial stability may play an important role in student retention. Students experiencing financial difficulties may be more likely to discontinue their studies, making tuition payment status a valuable feature for identifying at-risk students.

6.2.3. Age vs Dropout

In [16]:
```
plt.figure(figsize=(6,4))
sns.histplot(data=df, x="Age at enrollment", hue="Target", bins=20, kde=True)

plt.title("Age Distribution by Student Outcome")
plt.xlabel("Age at Enrollment")
plt.ylabel("Count")

plt.show()
```



The age distribution shows differences between students who graduate and those who drop out.

Younger students are more concentrated in the graduate group, with a clear peak in the late teens and early twenties. In contrast, dropout students are more widely distributed across age groups, with a higher representation among older students.

This suggests that older students may face additional challenges, such as work commitments or personal responsibilities, which could increase the likelihood of dropping out. While age is not the strongest predictor on its own, it appears to contribute to dropout risk when combined with other factors.

7. Data Preparation

Before training machine learning models, the dataset must be prepared. This stage includes removing variables that may introduce data leakage, separating the feature variables from the target variable, and splitting the dataset into training and testing sets.

Since the objective of this project is to predict student dropout early in the academic journey, features related to the second semester will be removed to ensure that the model only uses information that would realistically be available at the time of prediction.

7.1. Remove Second Semester Features (Data Leakage)

In [17]:
```
# Remove second semester features to avoid data leakage

df = df.drop(columns=[
  "Curricular units 2nd sem (credited)",
  "Curricular units 2nd sem (enrolled)",
  "Curricular units 2nd sem (evaluations)",
  "Curricular units 2nd sem (approved)",
  "Curricular units 2nd sem (grade)",
  "Curricular units 2nd sem (without evaluations)"
])

df.shape
```

Out[17]: (3630, 31)

7.2. Categorical Feature Encoding

Some features in the dataset are categorical but encoded as integers (e.g., course, nationality, parental occupation). These do not represent numerical relationships, so one-hot encoding was applied to convert them into binary features. This ensures that the model does not incorrectly assume any ordering between categories.

7.2.1. Define Feature Matrix (X) and Target (Y)

In [18]:
```
X = df.drop("Target", axis=1)
y = df["Target"]
```

7.2.2. Reduce High-Cardinality Categories

Some categorical variables contain many unique values. To reduce dimensionality, only the most frequent categories are retained, while less frequent ones are grouped into a single "Other" category.

In [19]:
```
# Function to group rare categories
def group_rare_categories(df, column, top_n=10):
  top_categories = df[column].value_counts().nlargest(top_n).index
  df[column] = df[column].apply(lambda x: x if x in top_categories else "Other")
  return df
```

```
# Columns with high cardinality
high_cardinality_cols = [
  "Course",
  "Nacionality",
  "Mothers occupation",
  "Fathers occupation",
  "Mothers qualification",
  "Fathers qualification"
]

# Apply grouping
for col in high_cardinality_cols:
  df = group_rare_categories(df, col, top_n=10)

X = df.drop("Target", axis=1)

print("Unique values after grouping:")
for col in high_cardinality_cols:
  print(col, ":", df[col].nunique())
```

```
Unique values after grouping:
Course : 11
Nacionality : 11
Mothers occupation : 11
Fathers occupation : 11
Mothers qualification : 11
Fathers qualification : 11
```

7.2.3. Define Categorical Columns

In [20]:
```
categorical_cols = [
  "Marital status",
  "Application mode",
  "Course",
  "Nacionality",
  "Mothers qualification",
  "Fathers qualification",
  "Mothers occupation",
  "Fathers occupation"
]
```

7.2.4. Apply One-Hot Encoding

After reducing category cardinality, one-hot encoding is applied to convert categorical variables into binary features. This ensures that machine learning models do not assume any ordinal relationship between categories.

In [21]:
```
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

print("New shape:", X.shape)
```

```
New shape: (3630, 104)
```

7.3. Correlation-Based Redundancy Check

Some features may contain overlapping information. Highly correlated variables can introduce redundancy, especially for linear models such as Logistic Regression, and may reduce model interpretability.

To investigate this, a correlation-based redundancy check was performed on the preprocessed dataset. Pairs of features with very high correlation were identified, and one feature from each pair was considered for removal. This step helps simplify the feature space while preserving the most relevant information.

7.3.1. Compute Feature Correlation Matrix

In [22]:
```
corr_matrix = X.corr().abs()
```

7.3.2. Isolate the Upper Triangle

In [23]:
```
upper_triangle = corr_matrix.where(
  np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
)
```

7.3.3. Identify Highly Correlated Features

In [24]:
```
high_corr_features = [
  column for column in upper_triangle.columns
  if any(upper_triangle[column] > 0.90)
]

print("Highly correlated features to consider removing:")
print(high_corr_features)
```

```
Highly correlated features to consider removing:
[]
```

7.3.4. Display Highly Correlated Feature Pairs

In [25]:
```
high_corr_pairs = []

for col in upper_triangle.columns:
    for row in upper_triangle.index:
        corr_value = upper_triangle.loc[row, col]
        if pd.notnull(corr_value) and corr_value > 0.90:
            high_corr_pairs.append((row, col, corr_value))

high_corr_df = pd.DataFrame(
    high_corr_pairs,
    columns=["Feature 1", "Feature 2", "Correlation"]
).sort_values(by="Correlation", ascending=False)

high_corr_df
```

Out[25]:

	Feature 1	Feature 2	Correlation

7.3.5. Interpretation

The correlation-based redundancy check did not identify any feature pairs with correlation above the selected threshold of **0.90**. This suggests that, after removing second-semester variables and simplifying high-cardinality categorical features, the remaining predictors do not exhibit extreme pairwise redundancy.

As a result, no features were removed at this stage. The full feature set was retained for the next step of the analysis, where statistical feature selection is applied to identify the most relevant predictors.

7.4. Train/Test Split

In [26]:
```
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training set:", X_train.shape)
print("Test set:", X_test.shape)
```

```
Training set: (2904, 104)
Test set: (726, 104)
```

7.5. Scale the Data

In [27]:
```
# Initialize scaler
scaler = StandardScaler()

# Fit scaler on training data
X_train_scaled = scaler.fit_transform(X_train)

# Apply same transformation to test data
X_test_scaled = scaler.transform(X_test)
```

7.6. Statistical Feature Selection (SelectKBest)

Although the redundancy check did not identify any highly correlated feature pairs for removal, the dataset still contains a relatively large number of predictors. To further reduce dimensionality, statistical feature selection is applied.

`SelectKBest` is used to rank features according to their individual relationship with the target variable. In this analysis, the **ANOVA F-test (f_classif)** is used to identify the features that are most strongly associated with student dropout.

To avoid data leakage, feature selection is fitted only on the training set and then applied to the test set using the same transformation.

7.6.1. Import Feature Selection Method

In [28]:
```
from sklearn.feature_selection import SelectKBest, f_classif
```

7.6.2. Remove Constant Features

In [29]:
```
constant_cols = [col for col in X_train.columns if X_train[col].nunique() <= 1]

print("Constant features:", constant_cols)

X_train_fs = X_train.drop(columns=constant_cols)
X_test_fs = X_test.drop(columns=constant_cols)
```

```
Constant features: ['Application mode_27']
```

7.6.3. Apply SelectKBest (Multiple Values of K)

In [30]:
```
k_values = [20, 30, 40]
kbest_results = []

for k in k_values:
    selector = SelectKBest(score_func=f_classif, k=k)

    X_train_sel = selector.fit_transform(X_train_fs, y_train)
    X_test_sel = selector.transform(X_test_fs)

    scaler_sel = StandardScaler()
    X_train_sel_scaled = scaler_sel.fit_transform(X_train_sel)
    X_test_sel_scaled = scaler_sel.transform(X_test_sel)

    model = LogisticRegression(
        max_iter=1000,
        class_weight='balanced'
    )

    model.fit(X_train_sel_scaled, y_train)

    y_pred_sel = model.predict(X_test_sel_scaled)
    y_probs_sel = model.predict_proba(X_test_sel_scaled)[:, 1]

    kbest_results.append({
        "K": k,
        "Accuracy": accuracy_score(y_test, y_pred_sel),
        "Precision": precision_score(y_test, y_pred_sel),
        "Recall": recall_score(y_test, y_pred_sel),
        "F1 Score": f1_score(y_test, y_pred_sel),
        "ROC-AUC": roc_auc_score(y_test, y_probs_sel)
    })

kbest_results_df = pd.DataFrame(kbest_results)
kbest_results_df
```

Out[30]:

	K	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	20	0.877410	0.815534	0.887324	0.849916	0.947326
1	30	0.885675	0.827362	0.894366	0.859560	0.949063
2	40	0.874656	0.818482	0.873239	0.844974	0.944005

*Multiple values of `k` were evaluated to determine how many features should be retained. Among the tested values, **k = 30** produced the highest F1-score and ROC-AUC, while also achieving the highest recall. Therefore, the top 30 features were selected for the final feature-selection experiment.*

7.6.3. Select the Best Value of K

In [31]:

```
best_k = 30

selector = SelectKBest(score_func=f_classif, k=best_k)

X_train_sel = selector.fit_transform(X_train_fs, y_train)
X_test_sel = selector.transform(X_test_fs)

print("Original number of features:", X_train_fs.shape[1])
print("Selected number of features:", X_train_sel.shape[1])
```

```
Original number of features: 103
Selected number of features: 30
```

7.6.4. Display Selected Features

In [32]:

```
selected_features = X_train_fs.columns[selector.get_support()]

print(f"Selected features for k={best_k}:")
for feature in selected_features:
  print(feature)
```

```
Selected features for k=30:
Application order
Daytime/evening attendance
Previous qualification (grade)
Admission grade
Displaced
Debtor
Tuition fees up to date
Gender
Scholarship holder
Age at enrollment
Curricular units 1st sem (enrolled)
Curricular units 1st sem (approved)
Curricular units 1st sem (grade)
Curricular units 1st sem (without evaluations)
Marital status_2
Marital status_4
Application mode_7
Application mode_39
Course_9070
Course_9238
Course_9500
Course_9991
Course_Other
Mothers qualification_34
Mothers qualification_37
Mothers qualification_38
Fathers qualification_34
Fathers qualification_38
Fathers qualification_Other
Mothers occupation_90
```

7.6.5. Display Feature Scores

In [33]:
```
feature_scores = pd.DataFrame({
  "Feature": X_train_fs.columns,
  "Score": selector.scores_
}).sort_values(by="Score", ascending=False)

feature_scores.head(30)
```

Out[33]:

	Feature	Score
16	Curricular units 1st sem (approved)	1287.563132
17	Curricular units 1st sem (grade)	1073.493875
8	Tuition fees up to date	688.692019
10	Scholarship holder	334.808809
7	Debtor	227.742247
11	Age at enrollment	213.169297
9	Gender	203.520695
36	Application mode_39	195.533046
48	Course_9500	129.254719
52	Course_Other	128.234140
14	Curricular units 1st sem (enrolled)	81.209477
4	Admission grade	65.298471
69	Mothers qualification_34	57.917119
3	Previous qualification (grade)	50.906276
51	Course_9991	44.594006
79	Fathers qualification_34	44.260915
29	Application mode_7	35.131854
46	Course_9238	34.570332
5	Displaced	34.555454
91	Mothers occupation_90	32.930703
0	Application order	24.679412
81	Fathers qualification_38	23.984177
1	Daytime/evening attendance	20.760167
22	Marital status_2	19.910093
43	Course_9070	13.279606
71	Mothers qualification_38	13.199364
24	Marital status_4	13.143967
82	Fathers qualification_Other	13.114934
70	Mothers qualification_37	12.980182
18	Curricular units 1st sem (without evaluations)	11.973102

7.6.5.1. Interpreting the Selected Features

The `SelectKBest` method ranks features according to how strongly each feature is individually associated with the target variable in the **training data**. In this analysis, **the ANOVA F-test (f_classif)** is used, which evaluates whether the mean value of a feature differs significantly between the target classes.

For continuous features, this means that features such as grades or approved curricular units are selected when students with different values show clearly different dropout patterns. For one-hot encoded categorical variables, such as `Course_9500` or `Application mode_39`, the score reflects whether belonging to that specific category is associated with a dropout pattern that differs from the rest of the dataset.

It is important to note that this does not mean that a selected category directly causes dropout. Instead, it means that, within this dataset, that category provides useful statistical separation between dropout and graduate cases. Such features may reflect broader differences in programme difficulty, student background, course structure, or other contextual factors.

In [34]:
```
# Selected categorical dummy features
selected_categorical_features = [
  feature for feature in selected_features
  if "_" in feature and not feature.startswith("Curricular")
]

print("Selected categorical dummy features:")
for feature in selected_categorical_features:
  print(feature)
```

```
Selected categorical dummy features:
Marital status_2
Marital status_4
Application mode_7
Application mode_39
Course_9070
Course_9238
Course_9500
Course_9991
Course_Other
Mothers qualification_34
Mothers qualification_37
Mothers qualification_38
Fathers qualification_34
Fathers qualification_38
Fathers qualification_Other
Mothers occupation_90
```

In [35]:
```
# Investigate dropout rate and size for original grouped categorical variables
categorical_analysis = {}

for col in high_cardinality_cols + ["Application mode", "Marital status"]:
  summary = df.groupby(col)["Target"].agg(["count", "mean"]).sort_values(by="mea
  summary["Dropout Rate (%)"] = summary["mean"] * 100
  categorical_analysis[col] = summary[["count", "Dropout Rate (%)"]]

categorical_analysis["Course"].head(15)
```

Out[35]:

Course	count	Dropout Rate (%)
9991	214	63.551402
Other	797	58.218319
9147	272	49.264706
9254	211	45.497630
9670	220	43.181818
8014	194	36.597938
9085	262	34.351145
9773	297	34.006734
9070	184	27.717391
9238	313	20.766773
9500	666	17.717718

In [36]:

```
print("Course categories selected in top 30:")
print(categorical_analysis["Course"].loc[[9070, 9238, 9500, 9991, "Other"]])

print("\nApplication mode categories selected in top 30:")
print(categorical_analysis["Application mode"].loc[[7, 39]])

print("\nMothers qualification categories selected in top 30:")
print(categorical_analysis["Mothers qualification"].loc[[34, 37, 38]])

print("\nFathers qualification categories selected in top 30:")
print(categorical_analysis["Fathers qualification"].loc[[34, 38, "Other"]])

print("\nMothers occupation categories selected in top 30:")
print(categorical_analysis["Mothers occupation"].loc[[90]])
```

```
Course categories selected in top 30:
        count  Dropout Rate (%)
Course
9070      184         27.717391
9238      313         20.766773
9500      666         17.717718
9991      214         63.551402
Other     797         58.218319

Application mode categories selected in top 30:
                  count  Dropout Rate (%)
Application mode
7                   132         64.393939
39                  664         65.512048

Mothers qualification categories selected in top 30:
                       count  Dropout Rate (%)
Mothers qualification
34                       127         75.590551
37                       840         45.595238
38                       471         29.723992

Fathers qualification categories selected in top 30:
                       count  Dropout Rate (%)
Fathers qualification
34                       109         74.311927
38                       575         29.043478
Other                     75         61.333333

Mothers occupation categories selected in top 30:
                    count  Dropout Rate (%)
Mothers occupation
90                     70         72.857143
```

7.6.5.2. Investigating Selected Categorical Features

In [37]:

```python
# Create a summary table for the selected dummy features
selected_dummy_summary = []

for feature in selected_categorical_features:
  original_col, category = feature.rsplit("_", 1)

  # Convert numeric-looking category labels back to integers where possible
  try:
      category_lookup = int(category)
  except ValueError:
      category_lookup = category

  if original_col in categorical_analysis and category_lookup in categorical_ana
      row = categorical_analysis[original_col].loc[category_lookup]

      selected_dummy_summary.append({
          "Dummy Feature": feature,
          "Original Variable": original_col,
          "Category": category_lookup,
          "Count": row["count"],
          "Dropout Rate (%)": row["Dropout Rate (%)"],
          "SelectKBest Score": feature_scores.loc[
              feature_scores["Feature"] == feature, "Score"
```

```
            ].values[0]
        })

selected_dummy_summary_df = pd.DataFrame(selected_dummy_summary).sort_values(
    by="SelectKBest Score",
    ascending=False
)

selected_dummy_summary_df
```

Out[37]:

	Dummy Feature	Original Variable	Category	Count	Dropout Rate (%)	SelectKBest Score
3	Application mode_39	Application mode	39	664.0	65.512048	195.533046
6	Course_9500	Course	9500	666.0	17.717718	129.254719
8	Course_Other	Course	Other	797.0	58.218319	128.234140
9	Mothers qualification_34	Mothers qualification	34	127.0	75.590551	57.917119
7	Course_9991	Course	9991	214.0	63.551402	44.594006
12	Fathers qualification_34	Fathers qualification	34	109.0	74.311927	44.260915
2	Application mode_7	Application mode	7	132.0	64.393939	35.131854
5	Course_9238	Course	9238	313.0	20.766773	34.570332
15	Mothers occupation_90	Mothers occupation	90	70.0	72.857143	32.930703
13	Fathers qualification_38	Fathers qualification	38	575.0	29.043478	23.984177
0	Marital status_2	Marital status	2	327.0	54.740061	19.910093
4	Course_9070	Course	9070	184.0	27.717391	13.279606
11	Mothers qualification_38	Mothers qualification	38	471.0	29.723992	13.199364
1	Marital status_4	Marital status	4	75.0	56.000000	13.143967
14	Fathers qualification_Other	Fathers qualification	Other	75.0	61.333333	13.114934
10	Mothers qualification_37	Mothers qualification	37	840.0	45.595238	12.980182

Interpretation

The selected categorical dummy features tend to represent categories with dropout rates that differ clearly from the overall dataset pattern. This helps explain why they received relatively high `SelectKBest` scores.

For example, `Application mode_39` has both a **large** number of observations and a **high** dropout rate, which makes it a strong separator between dropout and graduate cases. Similarly, categories such as `Course_9500` and `Course_9238` were selected because they show much **lower** dropout rates, while categories such as `Course_Other` and `Course_9991` show higher dropout rates.

This shows that `SelectKBest` selects categories that provide strong statistical separation in either direction. In other words, a category can be relevant because it is associated with either lower or higher dropout risk. These results should still be interpreted as associations in the dataset, not as direct causes of dropout.

7.6.5.3. Why Were Some Categories More Relevant Than Others?

Some categories were more relevant than others because they showed a clearer difference in dropout rate and had enough observations for that difference to be statistically meaningful.

This explains why categories such as `Application mode_39`, `Course_9500`, and `Course_Other` received especially high scores. They combine noticeable outcome differences with relatively large group sizes. In contrast, categories with weaker dropout differences or fewer observations are less likely to be selected.

This also shows that `SelectKBest` does not only select categories associated with higher dropout risk. It can also select categories associated with lower dropout risk when they provide strong separation between dropout and graduate cases.

7.6.5.4. Investigating Top Continuous and Binary Features

In [38]:
```
top_non_dummy_features = [
  "Curricular units 1st sem (approved)",
  "Curricular units 1st sem (grade)",
  "Tuition fees up to date",
  "Scholarship holder",
  "Debtor",
  "Age at enrollment"
]

continuous_binary_summary = []

for feature in top_non_dummy_features:
  summary = df.groupby("Target")[feature].agg(["mean", "median"])
  continuous_binary_summary.append({
    "Feature": feature,
    "Graduate Mean (0)": summary.loc[0, "mean"],
    "Dropout Mean (1)": summary.loc[1, "mean"],
    "Graduate Median (0)": summary.loc[0, "median"],
    "Dropout Median (1)": summary.loc[1, "median"]
  })

continuous_binary_summary_df = pd.DataFrame(continuous_binary_summary)
continuous_binary_summary_df
```

Out[38]:

	Feature	Graduate Mean (0)	Dropout Mean (1)	Graduate Median (0)	Dropout Median (1)
0	Curricular units 1st sem (approved)	6.232232	2.551724	6.0	2.000000
1	Curricular units 1st sem (grade)	12.643655	7.256656	13.0	10.928571
2	Tuition fees up to date	0.986872	0.678395	1.0	1.000000
3	Scholarship holder	0.377999	0.094300	0.0	0.000000
4	Debtor	0.045722	0.219564	0.0	0.000000
5	Age at enrollment	21.783612	26.068966	19.0	23.000000

Interpretation

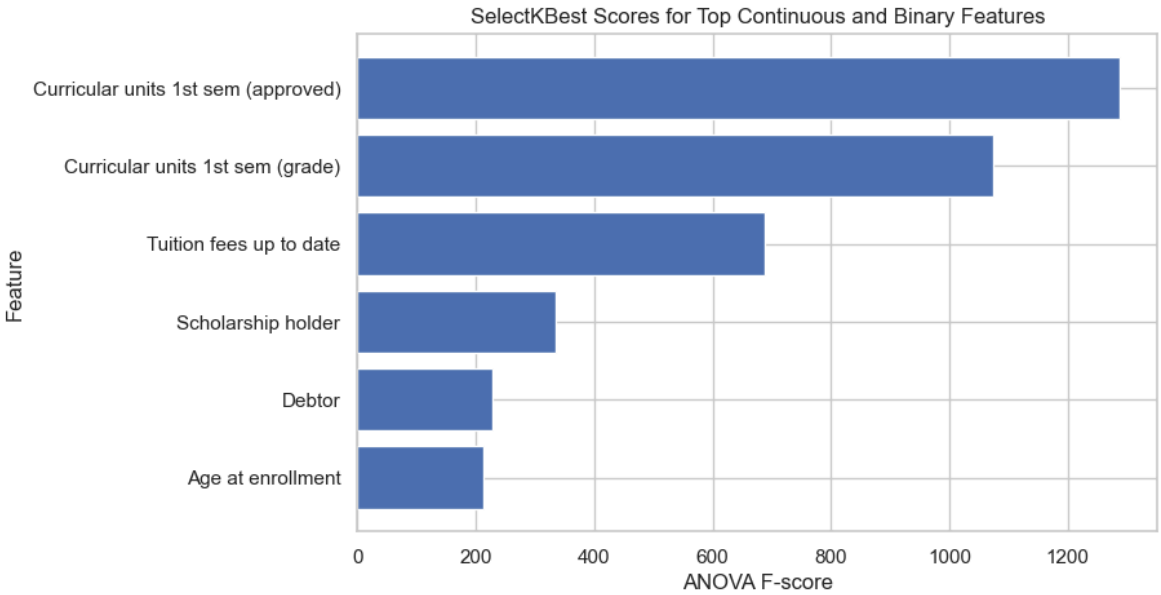
The results show clear differences between graduate and dropout students for the strongest non-categorical selected features.

Students who graduate complete more first-semester curricular units and achieve higher first-semester grades on average than students who drop out. Graduates are also more likely to have their tuition fees up to date, less likely to be debtors, and more likely to be scholarship holders. In addition, dropout students are older on average.

These differences help explain why these features received high SelectKBest scores: they provide strong individual separation between the two outcome groups.

In [39]:

```
selected_top_scores = feature_scores[
  feature_scores["Feature"].isin(top_non_dummy_features)
].sort_values(by="Score", ascending=True)

plt.figure(figsize=(8, 5))
plt.barh(selected_top_scores["Feature"], selected_top_scores["Score"])
plt.title("SelectKBest Scores for Top Continuous and Binary Features")
plt.xlabel("ANOVA F-score")
plt.ylabel("Feature")
plt.show()
```



7.6.5.5. Limitations of SelectKBest

Although `SelectKBest` is useful for identifying strong individual predictors, its results should be interpreted carefully.

First, it is a univariate method, which means that each feature is **evaluated independently** rather than in combination with other variables. A feature may have a low individual score but still become useful when combined with other predictors in a full model.

Second, one-hot encoded categorical variables are split into separate dummy features. This means that some individual categories may appear highly relevant even though the overall original variable is only moderately informative.

Finally, high feature scores reflect statistical association rather than causation. The selected features help separate dropout and graduate cases in this dataset, but they should not be interpreted as direct causes of student dropout.

7.6.6. Scale Selected Features

Because Logistic Regression is sensitive to feature scale, the selected features should now be standardized.

In [40]:
```
scaler_sel = StandardScaler()

X_train_sel_scaled = scaler_sel.fit_transform(X_train_sel)
X_test_sel_scaled = scaler_sel.transform(X_test_sel)
```

8. Model Experiments

8.1. Baseline Model — Logistic Regression

The first model implemented in this analysis is `Logistic Regression`. Logistic Regression is commonly used for binary classification problems and provides a simple and interpretable baseline for evaluating predictive performance.

The goal of this step is to establish an initial benchmark that can later be compared with more complex machine learning models such as **Random Forest** and **Gradient Boosting**.

8.1.1. Train the Model

In [41]:
```
# Initialize the model
log_model = LogisticRegression(
  max_iter=1000,
  class_weight='balanced'
)

# Train the model
log_model.fit(X_train_scaled, y_train)
```

Out[41]:
```
▾ LogisticRegression
▸ Parameters
```

8.1.2. Cross-Validation

In [42]:
```
log_cv = cross_val_score(
  log_model,
  X_train_scaled,
  y_train,
  cv=5,
  scoring="f1"
)

print("Logistic Regression CV F1:", log_cv)
print("Average CV F1:", log_cv.mean())
```

```
Logistic Regression CV F1: [0.83660131 0.8453159  0.80786026 0.83916084 0.8453159
]
Average CV F1: 0.8348508433275965
```

8.1.3. Make Predictions

In [43]:
```
# Predict on the test set
y_pred = log_model.predict(X_test_scaled)
```

8.1.4. Evaluate the Model

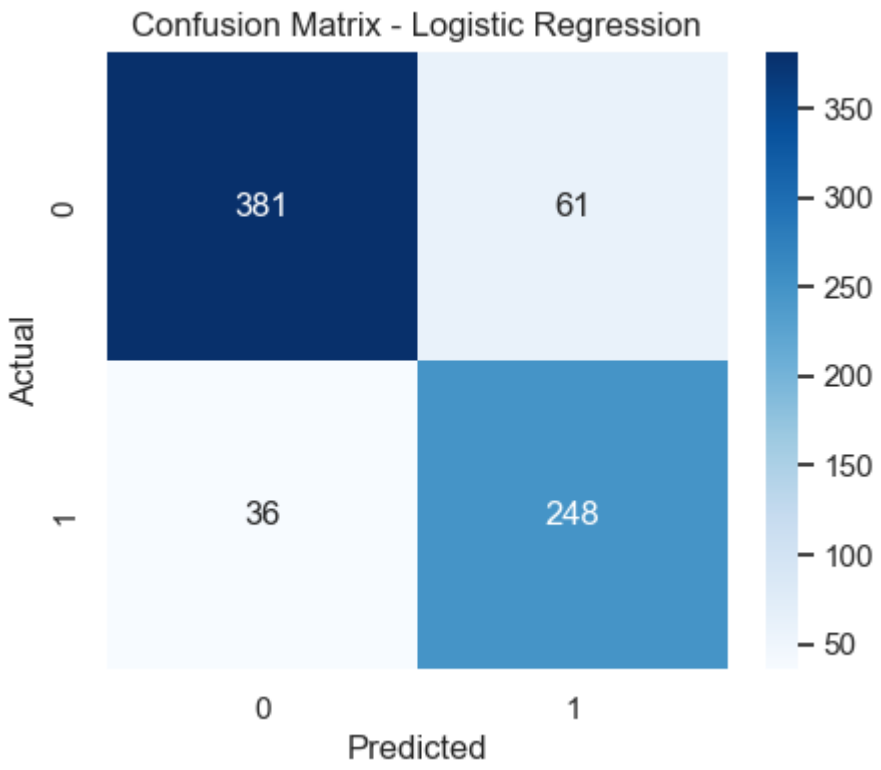
In [44]:
```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

# Calculate ROC-AUC
```

```
y_probs = log_model.predict_proba(X_test_scaled)[:, 1]

roc_auc = roc_auc_score(y_test, y_probs)

print("ROC-AUC Score:", roc_auc)
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

```
ROC-AUC Score: 0.9424431202600217
Accuracy: 0.8663911845730028
Precision: 0.8025889967637541
Recall: 0.8732394366197183
F1 Score: 0.836424957841484
```

8.1.5. Confusion Matrix

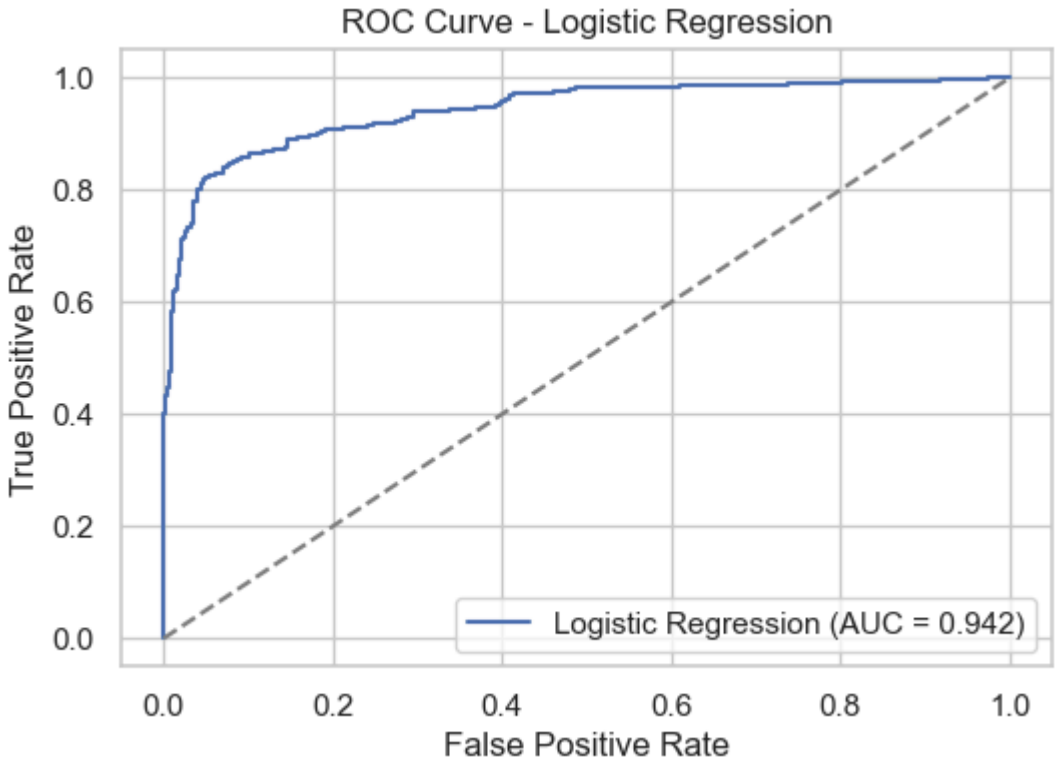
In [45]:
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Logistic Regression")

plt.show()
```



8.1.6. ROC Curve

In [46]:
```
# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_probs)
```

```
plt.figure(figsize=(6,4))

plt.plot(fpr, tpr, label=f"Logistic Regression (AUC = {roc_auc:.3f})")
plt.plot([0,1], [0,1], linestyle="--", color="gray")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Logistic Regression")
plt.legend()

plt.show()
```



ROC Curve Interpretation

The ROC curve evaluates the ability of the Logistic Regression model to distinguish between students who will drop out and those who will graduate. The model achieves an AUC score of **0.942**, indicating excellent classification performance.

The curve rises steeply near the origin, showing that the model can achieve a high true positive rate while keeping the false positive rate relatively low. This suggests that the model is effective at identifying students at risk of dropping out.

Overall, the high `ROC-AUC` score confirms that the model has strong discriminative power and can reliably differentiate between the two student outcome classes.

8.1.7. Baseline Model Performance Interpretation

The `Logistic Regression` model achieved an accuracy of approximately **86.64%**, indicating that the majority of student outcomes were correctly classified. This result indicates that a relatively simple linear model is able to capture much of the relationship between the predictors and student dropout.

The confusion matrix shows that the model correctly predicted **381** students who graduated and **248** students who dropped out. However, **61** students were incorrectly predicted as dropouts, while **36** dropout cases were incorrectly classified as graduates.

The recall score of **0.87** indicates that the model successfully identifies approximately **87%** of students who eventually drop out. This is particularly important in the context of early intervention systems, where detecting at-risk students is a key objective.

Overall, the `Logistic Regression` model provides a strong baseline performance that will serve as a reference when evaluating more advanced models in the following sections.

8.1.8. Logistic Regression with Selected Features

In addition to the `Baseline Logistic Regression` model trained on the full feature set, a second `Logistic Regression` model is trained using the 30 features selected through the `SelectKBest` procedure described in Section 7.6.

This experiment evaluates whether reducing the dimensionality of the dataset improves model performance while keeping the model simpler and more interpretable.

In [47]:
```
# 8.1.8 Logistic Regression with Selected Features

log_model_sel = LogisticRegression(
  max_iter=1000,
  class_weight='balanced'
)

log_model_sel.fit(X_train_sel_scaled, y_train)
```

Out[47]:
```
▾ LogisticRegression
▸ Parameters
```

8.1.9. Evaluate Selected-Feature Logistic Regression

In [48]:
```
# Predict on the test set
y_pred_sel = log_model_sel.predict(X_test_sel_scaled)

# Predict probabilities
y_probs_sel = log_model_sel.predict_proba(X_test_sel_scaled)[:, 1]

# Evaluation metrics
sel_accuracy = accuracy_score(y_test, y_pred_sel)
sel_precision = precision_score(y_test, y_pred_sel)
sel_recall = recall_score(y_test, y_pred_sel)
sel_f1 = f1_score(y_test, y_pred_sel)
sel_auc = roc_auc_score(y_test, y_probs_sel)

print("Accuracy:", sel_accuracy)
print("Precision:", sel_precision)
print("Recall:", sel_recall)
```

```
print("F1 Score:", sel_f1)
print("ROC-AUC:", sel_auc)
```

```
Accuracy: 0.8856749311294766
Precision: 0.8273615635179153
Recall: 0.8943661971830986
F1 Score: 0.8595600676818951
ROC-AUC: 0.9490631572238863
```

8.1.10. Compare Baseline vs Selected-Feature Logistic Regression

In [49]:

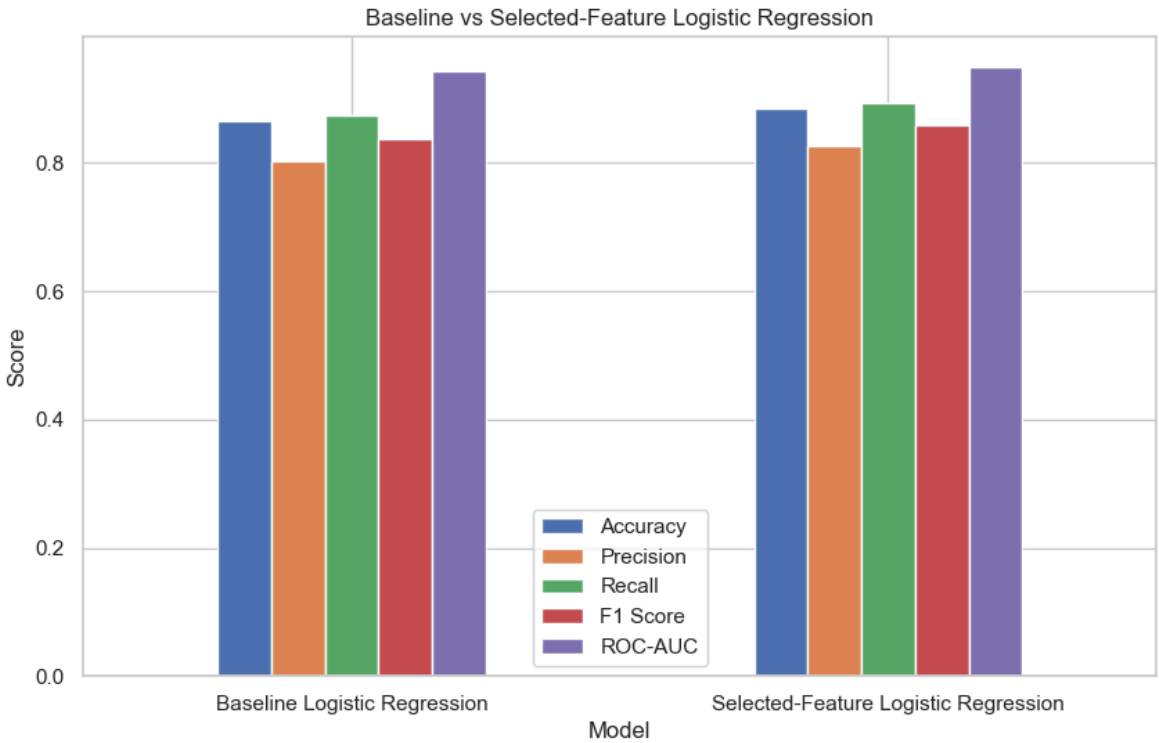
```
lr_comparison = pd.DataFrame({
  "Model": ["Baseline Logistic Regression", "Selected-Feature Logistic Regressio
  "Accuracy": [accuracy, sel_accuracy],
  "Precision": [precision, sel_precision],
  "Recall": [recall, sel_recall],
  "F1 Score": [f1, sel_f1],
  "ROC-AUC": [roc_auc, sel_auc]
})

lr_comparison
```

Out[49]:

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	Baseline Logistic Regression	0.866391	0.802589	0.873239	0.836425	0.942443
1	Selected-Feature Logistic Regression	0.885675	0.827362	0.894366	0.859560	0.949063

In [50]:

```
lr_comparison.set_index("Model").plot(kind="bar", figsize=(10,6))

plt.title("Baseline vs Selected-Feature Logistic Regression")
plt.ylabel("Score")
plt.xticks(rotation=0)

plt.show()
```



8.1.11. Interpretation

The Logistic Regression model trained on the 30 selected features outperformed the baseline Logistic Regression model trained on the full feature set. The selected-feature model achieved higher accuracy, precision, recall, F1-score, and ROC-AUC.

In particular, the F1-score improved from 0.836 to 0.860, while recall increased from 0.873 to 0.894. This indicates that the reduced feature set was more effective at identifying students at risk of dropping out.

These results suggest that feature selection removed weaker or noisier predictors and allowed the model to focus on the most informative variables. As a result, the selected-feature Logistic Regression model provides a better balance between predictive performance, simplicity, and interpretability.

8.2. Random Forest Model

`Random Forest` is an ensemble learning algorithm that builds multiple decision trees and aggregates their predictions. It is capable of capturing non-linear relationships between variables and often performs well on tabular datasets.

8.2.1. Train the Model

In [51]:
```
# Initialize the model
rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight='balanced'
)
```

```
# Train the model
rf_model.fit(X_train, y_train)
```

Out[51]:

```
▾ RandomForestClassifier
▸ Parameters
```

8.2.2. Cross-Validation

Cross-validation is used to evaluate how well the model generalizes to unseen data by training and validating the model across multiple splits of the dataset.

In [52]:

```
cv_scores = cross_val_score(
    rf_model,
    X_train,
    y_train,
    cv=5,
    scoring="f1"
)

print("Cross-validation F1 scores:", cv_scores)
print("Average CV F1:", cv_scores.mean())
```

```
Cross-validation F1 scores: [0.85125858 0.84382284 0.81516588 0.83840749 0.851764
71]
Average CV F1: 0.840083900372669
```

8.2.3. Make Predictions

In [53]:

```
# Predict on the test set
rf_pred = rf_model.predict(X_test)
```

8.2.4. Evaluate the Model

In [54]:

```
rf_accuracy = accuracy_score(y_test, rf_pred)
rf_precision = precision_score(y_test, rf_pred)
rf_recall = recall_score(y_test, rf_pred)
rf_f1 = f1_score(y_test, rf_pred)

print("Accuracy:", rf_accuracy)
print("Precision:", rf_precision)
print("Recall:", rf_recall)
print("F1 Score:", rf_f1)
```

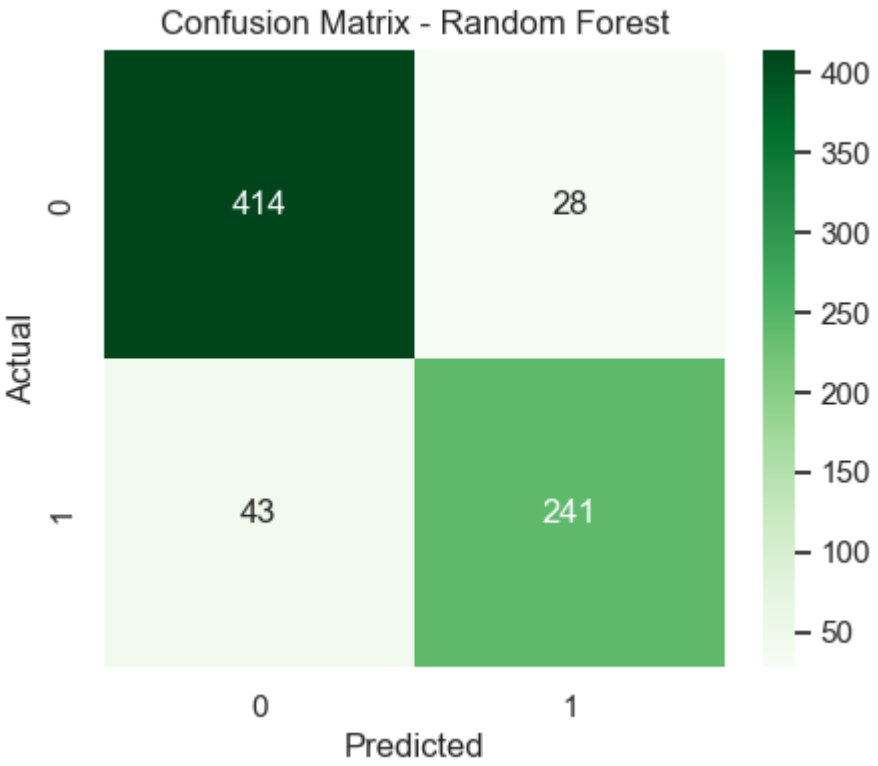
```
Accuracy: 0.9022038567493113
Precision: 0.895910780669145
Recall: 0.8485915492957746
F1 Score: 0.8716094032549728
```

8.2.5. Confusion Matrix

In [55]:

```
rf_cm = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(5,4))
```

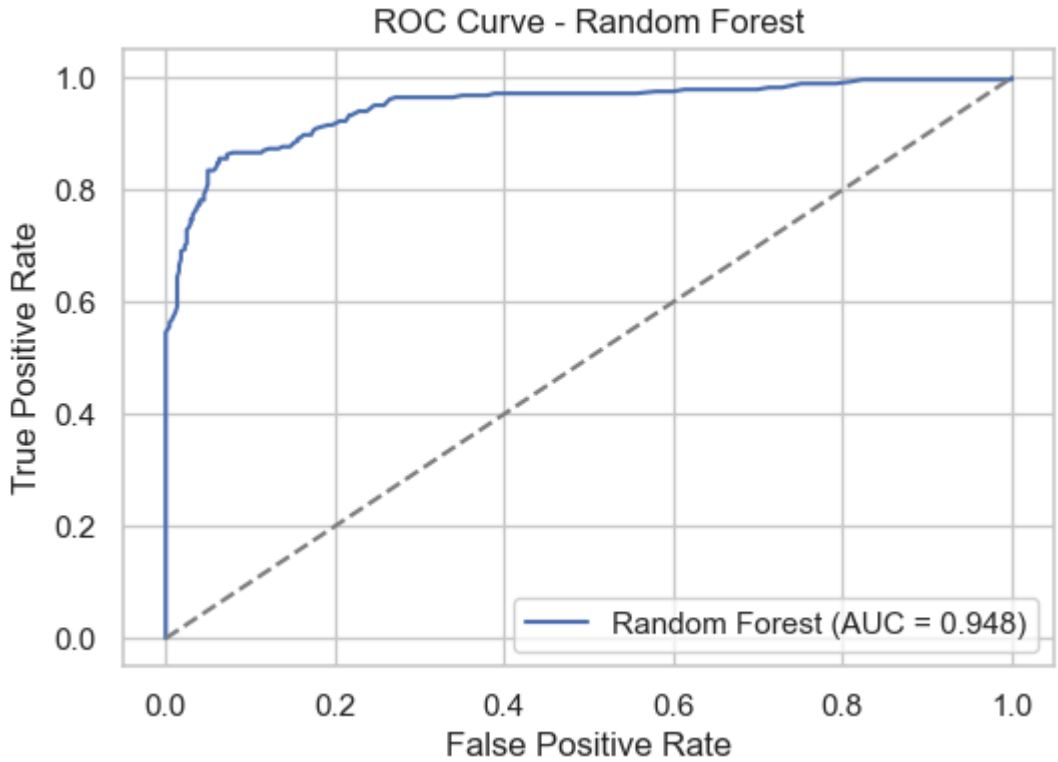
```
sns.heatmap(rf_cm, annot=True, fmt="d", cmap="Greens")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Random Forest")

plt.show()
```



8.2.6. ROC Curve

In [56]:
```
# Calculate predicted probabilities
rf_probs = rf_model.predict_proba(X_test)[:, 1]

# Calculate ROC-AUC score
rf_auc = roc_auc_score(y_test, rf_probs)

print("ROC-AUC Score:", rf_auc)

# Compute ROC curve
rf_fpr, rf_tpr, thresholds = roc_curve(y_test, rf_probs)

plt.figure(figsize=(6,4))

plt.plot(rf_fpr, rf_tpr, label=f"Random Forest (AUC = {rf_auc:.3f})")
plt.plot([0,1], [0,1], linestyle="--", color="gray")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Random Forest")
plt.legend()

plt.show()
```

```
ROC-AUC Score: 0.9480554139315531
```



The ROC curve evaluates the ability of the Random Forest model to distinguish between students who drop out and those who graduate. The ROC-AUC score indicates strong classification performance, suggesting that the model effectively separates the two outcome classes.

8.2.7. Feature Importance

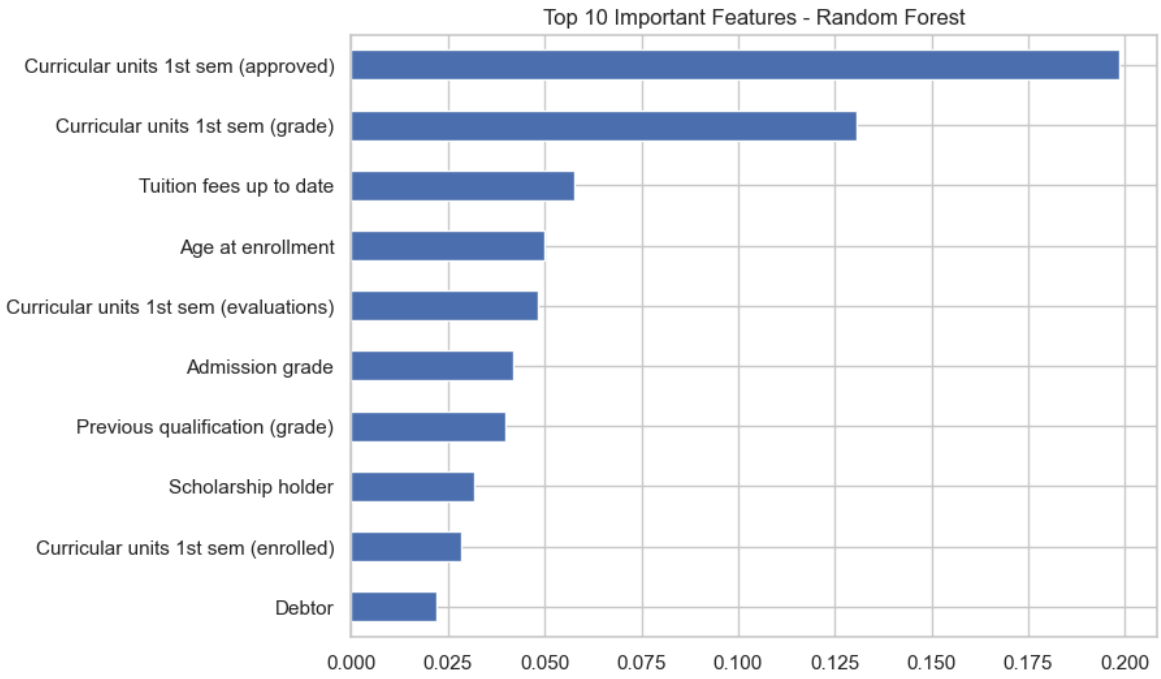
In [57]:
```
importances = pd.Series(
    rf_model.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print(importances.head(10))
```

```
Curricular units 1st sem (approved)       0.198529
Curricular units 1st sem (grade)          0.130527
Tuition fees up to date                   0.057580
Age at enrollment                         0.049842
Curricular units 1st sem (evaluations)    0.048139
Admission grade                           0.041954
Previous qualification (grade)            0.039993
Scholarship holder                        0.031742
Curricular units 1st sem (enrolled)       0.028341
Debtor                                    0.021987
dtype: float64
```

The high importance of first-semester approved curricular units suggests that early academic performance plays a dominant role in predicting student dropout risk.

In [58]:
```
plt.figure(figsize=(8,6))
importances.head(10).plot(kind="barh")

plt.title("Top 10 Important Features - Random Forest")
```

```
plt.gca().invert_yaxis()

plt.show()
```



8.2.8. Model Performance Interpretation

The `Random Forest` model achieved an accuracy of approximately **90.22%**, which is slightly higher than the `Logistic Regression baseline`. The model also achieved a precision score of **0.89**, a recall score of **0.84**, and an F1-score of **0.87**.

The confusion matrix shows that the model correctly classified **414** students who graduated and **241** students who dropped out. However, **43** dropout cases were incorrectly predicted as graduates, while **28** graduates were incorrectly classified as dropouts.

The recall score of **0.84** indicates that the model successfully identifies approximately **84%** of students who eventually drop out. This performance is comparable to the Logistic Regression model, suggesting that the relationships between the variables and the target outcome may be relatively `linear`.

The cross-validation results show relatively stable F1 scores across the five folds, with an average F1-score of approximately **0.84**. This indicates that the Random Forest model generalizes well to unseen data and does not appear to suffer from significant overfitting.

Feature importance analysis reveals that first-semester academic performance indicators are the strongest predictors of student dropout. In particular, the number of approved curricular units and first-semester grades appear to have the greatest influence on the model's predictions. These findings align with the domain research, which suggests that early academic performance is a strong indicator of student persistence.

Overall, the `Random Forest` model provides a slight improvement over the Logistic Regression baseline while offering additional insights into the relative importance of different features.

8.3. Gradient Boosting

Gradient Boosting is an ensemble learning algorithm that builds models sequentially, where each new model attempts to correct the errors made by the previous ones. Unlike `Random Forest`, which builds trees independently, `Gradient Boosting` improves predictions iteratively and often achieves strong performance on structured tabular datasets.

8.3.1 Train the Model

In [59]:
```
# Initialize the model
gb_model = GradientBoostingClassifier(
  n_estimators=200, #number of trees
  learning_rate=0.05, #how much each tree contributes to the final prediction
  random_state=42 #reproducibility
)

# Train the model
gb_model.fit(X_train, y_train)
```

Out[59]:
```
▾ GradientBoostingClassifier
▸ Parameters
```

8.3.2. Make Predictions

In [60]:
```
# Predict on the test set
gb_pred = gb_model.predict(X_test)
```

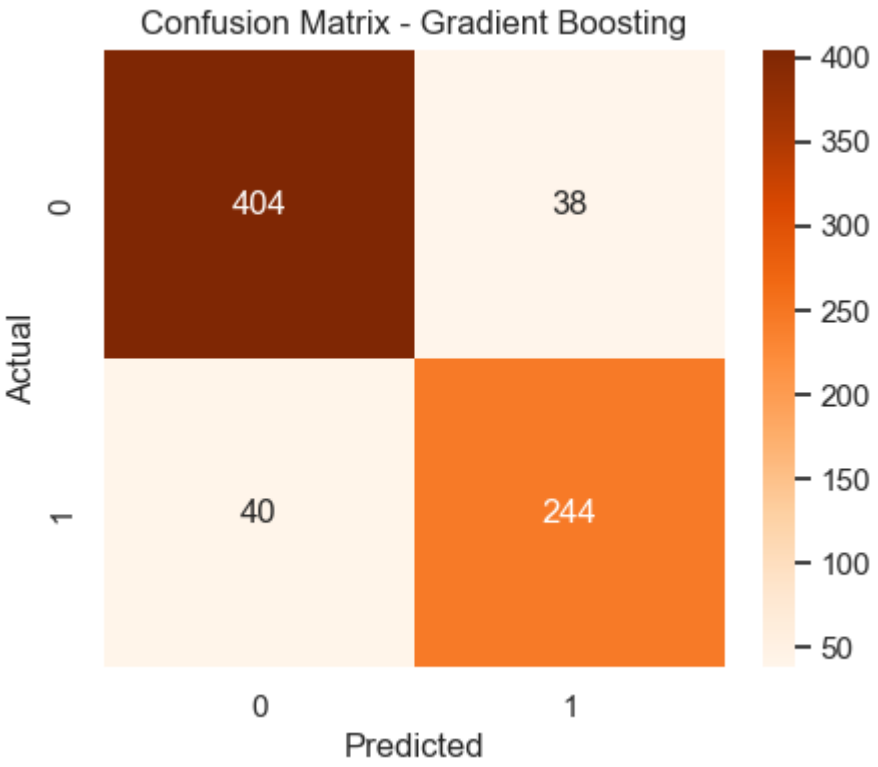
8.3.3 Evaluate the Model

In [61]:
```
gb_accuracy = accuracy_score(y_test, gb_pred)
gb_precision = precision_score(y_test, gb_pred)
gb_recall = recall_score(y_test, gb_pred)
gb_f1 = f1_score(y_test, gb_pred)

print("Accuracy:", gb_accuracy)
print("Precision:", gb_precision)
print("Recall:", gb_recall)
print("F1 Score:", gb_f1)
```

```
Accuracy: 0.8925619834710744
Precision: 0.8652482269503546
Recall: 0.8591549295774648
F1 Score: 0.8621908127208481
```

8.3.4 Confusion Matrix

In [62]:
```
gb_cm = confusion_matrix(y_test, gb_pred)

plt.figure(figsize=(5,4))
sns.heatmap(gb_cm, annot=True, fmt="d", cmap="Oranges")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Gradient Boosting")

plt.show()
```

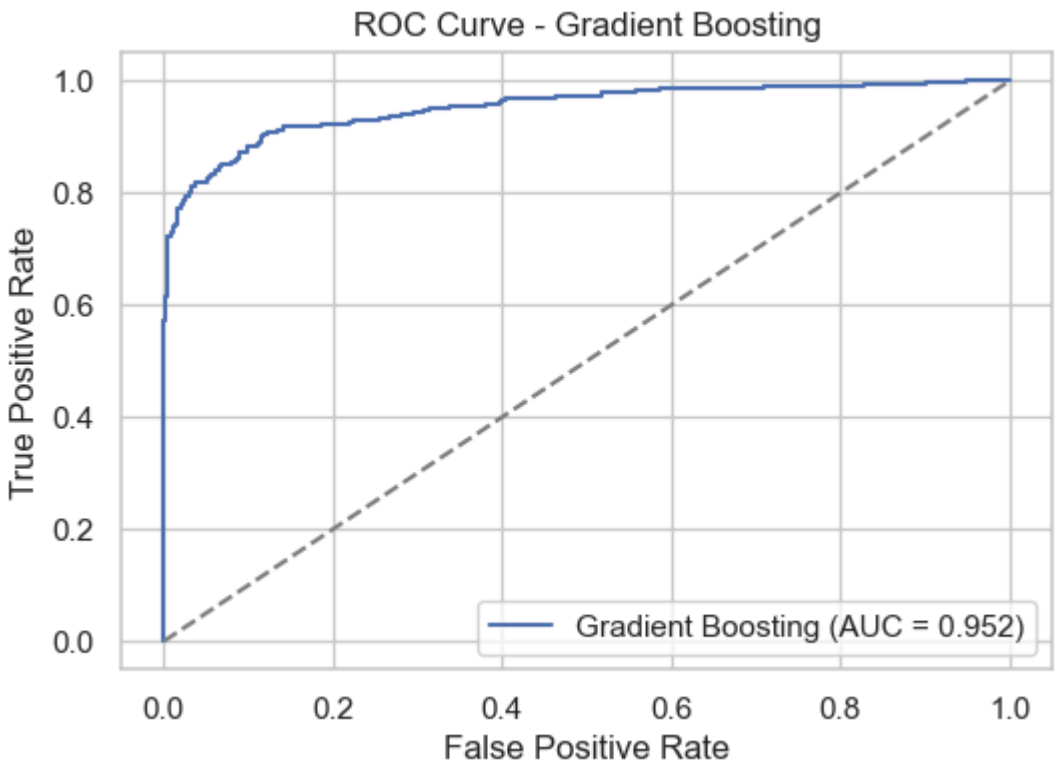


8.3.5. ROC Curve

In [63]:
```
# Calculate predicted probabilities
gb_probs = gb_model.predict_proba(X_test)[:, 1]

# Calculate ROC-AUC score
gb_auc = roc_auc_score(y_test, gb_probs)

print("ROC-AUC Score:", gb_auc)

# Compute ROC curve
gb_fpr, gb_tpr, thresholds = roc_curve(y_test, gb_probs)

plt.figure(figsize=(6,4))

plt.plot(gb_fpr, gb_tpr, label=f"Gradient Boosting (AUC = {gb_auc:.3f})")
plt.plot([0,1], [0,1], linestyle="--", color="gray")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
```

```
plt.title("ROC Curve - Gradient Boosting")
plt.legend()

plt.show()
```

```
ROC-AUC Score: 0.9520585048754061
```



The ROC curve shows the classification performance of the Gradient Boosting model across different decision thresholds. The ROC-AUC score indicates strong discriminative ability, meaning the model can effectively differentiate between students who are likely to drop out and those who are likely to graduate.

8.3.6. Model Performance Interpretation

The `Gradient Boosting` model achieved an accuracy of approximately **89.26%**, with a precision score of **0.86**, a recall score of **0.86**, and an F1-score of **0.86**. These results indicate that the model performs comparably to both the Logistic Regression and Random Forest models.

The confusion matrix shows that the model correctly classified **404** students who graduated and **244** students who dropped out. However, **38** dropout cases were incorrectly predicted as graduates, while **40** graduates were incorrectly classified as dropouts.

The recall score of **0.86** indicates that the model successfully identifies approximately **86%** of students who eventually drop out. Since the primary objective of this predictive system is early detection of at-risk students, recall remains an important metric for evaluating model effectiveness.

Compared with the other models tested in this analysis, the `Gradient Boosting` model performs very similarly. This suggests that the relationships between the

predictors and student dropout risk may be relatively well captured by both linear and ensemble methods.

Overall, the `Gradient Boosting` model provides strong predictive performance while maintaining balanced precision and recall, making it a competitive candidate for predicting student dropout risk.

9. Model Comparison (Original)

In this section, the performance of the different machine learning models is compared using the evaluation metrics calculated earlier. The goal is to determine which model provides the most reliable predictions for identifying students at risk of dropping out.

9.1. Comparison Table

In [64]:
```
results = pd.DataFrame({
  "Model": [
    "Baseline Logistic Regression",
    "Selected-Feature Logistic Regression",
    "Random Forest",
    "Gradient Boosting"
  ],
  "Accuracy": [
    accuracy,
    sel_accuracy,
    rf_accuracy,
    gb_accuracy
  ],
  "Precision": [
    precision,
    sel_precision,
    rf_precision,
    gb_precision
  ],
  "Recall": [
    recall,
    sel_recall,
    rf_recall,
    gb_recall
  ],
  "F1 Score": [
    f1,
    sel_f1,
    rf_f1,
    gb_f1
  ],
  "ROC-AUC": [
    roc_auc,
    sel_auc,
    rf_auc,
    gb_auc
  ]
})
```

```
results.sort_values(by="F1 Score", ascending=False)
```

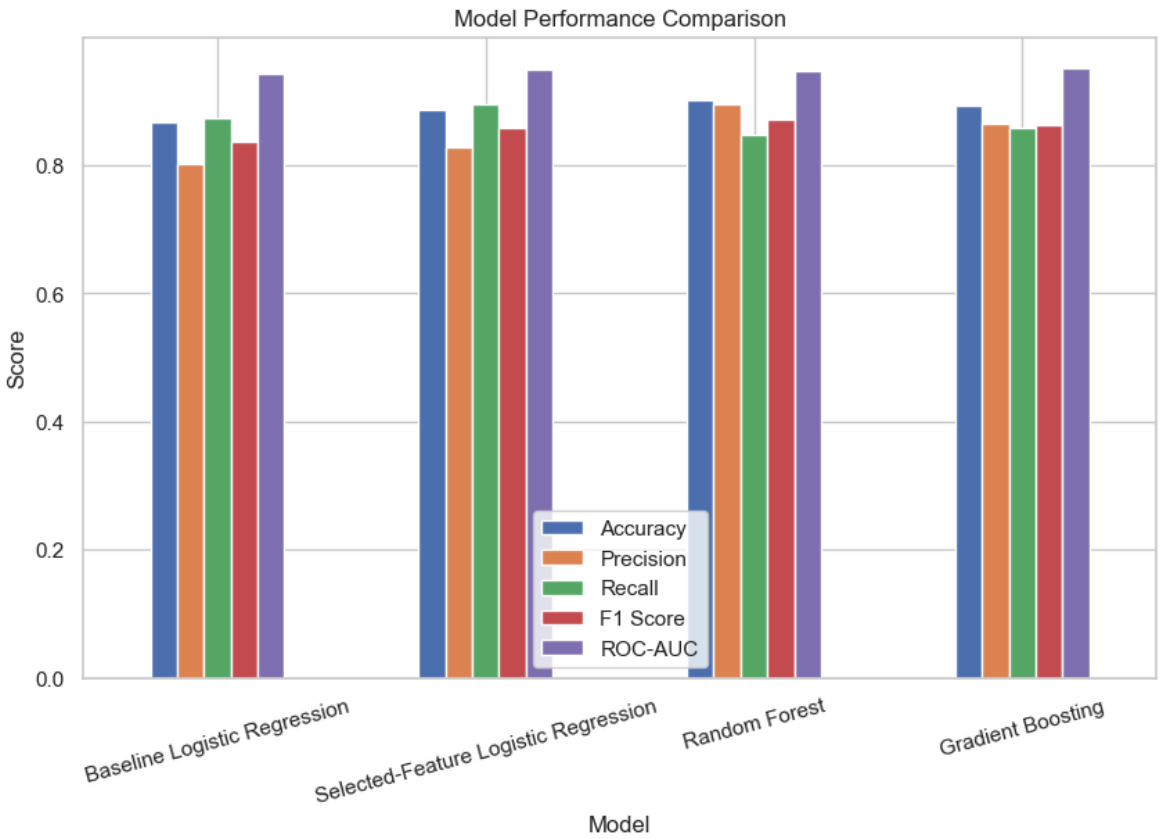
Out[64]:

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
2	Random Forest	0.902204	0.895911	0.848592	0.871609	0.948055
3	Gradient Boosting	0.892562	0.865248	0.859155	0.862191	0.952059
1	Selected-Feature Logistic Regression	0.885675	0.827362	0.894366	0.859560	0.949063
0	Baseline Logistic Regression	0.866391	0.802589	0.873239	0.836425	0.942443

9.2. Model Performance Visualization

In [65]:

```
results.set_index("Model").plot(kind="bar", figsize=(10,6))

plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.xticks(rotation=15)

plt.show()
```



9.3. Model Comparison Interpretation

The updated comparison shows that `Random Forest` achieved the highest F1-score, making it the strongest overall model in this experiment. Gradient Boosting also

performed strongly and achieved the highest ROC-AUC, indicating excellent discriminative ability across thresholds.

The `selected-feature Logistic Regression` model clearly improved over the baseline Logistic Regression model. After reducing the feature set to the 30 most informative predictors, Logistic Regression achieved higher accuracy, precision, recall, F1-score, and ROC-AUC.

Overall, Random Forest provides the best overall balance, while selected-feature Logistic Regression offers strong performance with greater simplicity and interpretability.

10. Ethical Feature Analysis

In this section, an additional experiment is conducted to evaluate the impact of removing sensitive demographic attributes from the model. Variables such as gender, nationality, and parental education or occupation may introduce social or cultural bias into predictive systems. To examine their influence, the models are retrained without these attributes and their performance is compared with the previous results.

According to the **EU Ethics Guidelines for Trustworthy AI**, predictive systems should avoid discriminatory outcomes and ensure fairness when demographic attributes are involved.

European Commission. (2019). Ethics Guidelines for Trustworthy AI. https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai

European Union. (2016). General Data Protection Regulation (GDPR). https://gdpr-info.eu/

10.1. Criteria for Identifying Sensitive Features

When building predictive systems that may influence educational decisions, it is important to consider the ethical implications of the features used for training machine learning models. Certain variables may represent **demographic identity** or **socioeconomic background**, which can introduce unfair bias in automated decision systems.

According to ethical guidelines for trustworthy AI and the principles of responsible data use, features may be considered sensitive if they:

• Represent **demographic identity** (e.g., gender or nationality)
• Represent **family socioeconomic background** (e.g., parents' education or occupation)
• Could act as **proxies for protected characteristics**
• May not be appropriate to use in real-world predictive systems due to **data protection regulations such as the EU General Data Protection Regulation (GDPR)**.

The goal of this analysis is therefore to identify whether any variables in the dataset fall into these categories and evaluate their potential impact on model predictions.

10.2. Identification of Potentially Sensitive Features

Based on the dataset structure, several variables describe either **demographic identity** or **family socioeconomic background** rather than academic performance. These variables may introduce ethical concerns if used directly in predictive models.

The following features are therefore identified as potentially sensitive:

- Gender
- Nacionality
- Mothers qualification
- Fathers qualification
- Mothers occupation
- Fathers occupation

These variables describe characteristics related to a student's demographic identity or family background rather than their academic performance. As a result, predictive models trained with these attributes may learn patterns that reflect social or economic inequalities instead of academic factors directly related to student success.

Sensitive Feature Justification

Feature	Category	Ethical Concern
Gender	Demographic identity	Potential risk of gender-related bias
Nacionality	Demographic identity	Possible bias toward majority nationality groups
Mothers qualification	Socioeconomic background	Proxy for family education level
Fathers qualification	Socioeconomic background	Proxy for family education level
Mothers occupation	Socioeconomic background	Proxy for household income
Fathers occupation	Socioeconomic background	Proxy for household income

These variables describe characteristics related to a student's background rather than their academic progress. Their use in predictive models may therefore raise ethical concerns regarding fairness and potential discrimination.

10.3. Data Governance and Privacy Considerations

In addition to fairness concerns, practical data governance constraints must also be considered when designing predictive systems for real-world educational environments.

Universities may not always be able to collect or use certain personal attributes without explicit student consent. Variables such as parental occupation, parental education, nationality, or gender may be considered sensitive personal information under regulations such as the **European General Data Protection Regulation (GDPR)**.

Furthermore, the principle of **data minimization** suggests that predictive models should rely only on information that is strictly necessary for the intended purpose. Since the objective of this model is to identify academic risk factors related to student performance, demographic and family background variables may not be required for accurate prediction.

For this reason, it is important to investigate whether similar predictive performance can be achieved without relying on these sensitive attributes.

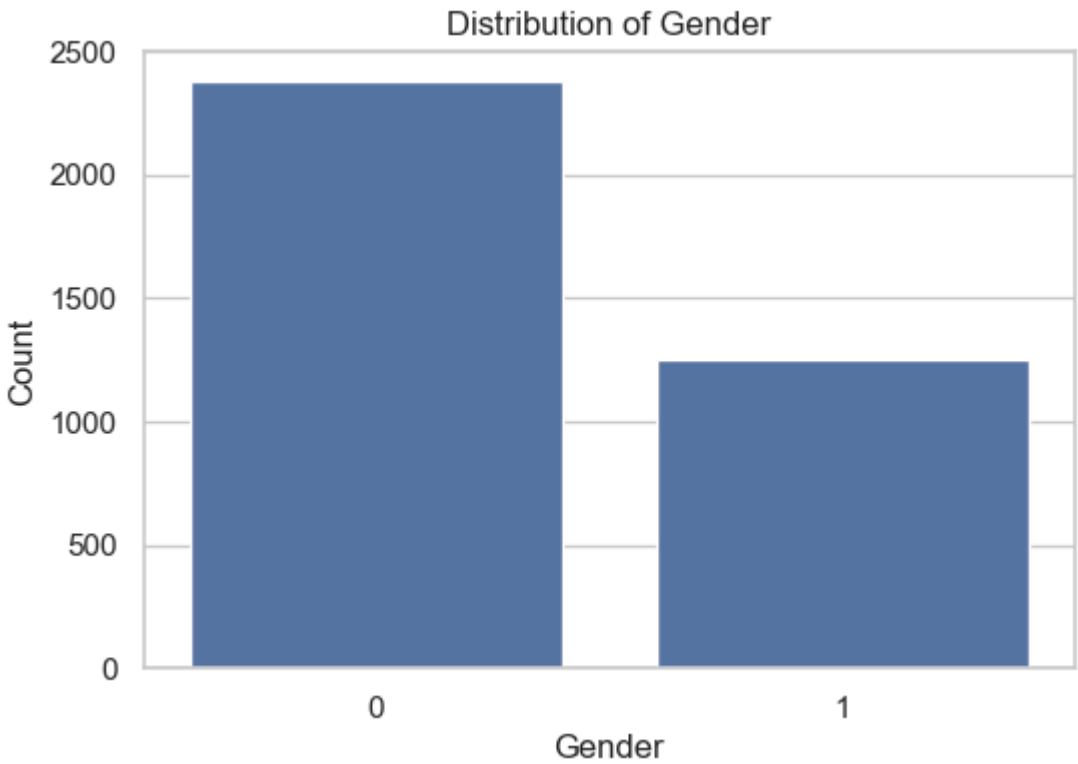
10.4. Distribution of Sensitive Features

In [66]:
```
sensitive_features = [
  "Gender",
  "Nacionality",
  "Mothers qualification",
  "Fathers qualification",
  "Mothers occupation",
  "Fathers occupation"
]

for feature in sensitive_features:
  print(f"\nTop categories for {feature}:")
  print(df[feature].value_counts().head(10))
```

```
Top categories for Gender:
Gender
0    2381
1    1249
Name: count, dtype: int64

Top categories for Nacionality:
Nacionality
1        3544
41         32
22         12
Other      10
26          9
6           8
24          5
103         3
11          3
101         2
Name: count, dtype: int64

Top categories for Mothers qualification:
Mothers qualification
1        865
37       840
19       777
38       471
3        337
34       127
Other     64
2         60
12        39
4         34
Name: count, dtype: int64

Top categories for Fathers qualification:
Fathers qualification
37       1010
19        785
1         732
38        575
3         220
34        109
Other      75
2          43
12         34
4          32
Name: count, dtype: int64

Top categories for Mothers occupation:
Mothers occupation
9        1313
4         670
5         436
3         272
2         240
7         224
0         143
Other      98
1          87
6          77
```

```
Name: count, dtype: int64

Top categories for Fathers occupation:
Fathers occupation
9        847
7        555
5        409
3        321
4        312
8        263
Other    245
10       212
6        194
2        147
Name: count, dtype: int64
```

In [67]:
```
plt.figure(figsize=(6,4))
sns.countplot(x="Gender", data=df)

plt.title("Distribution of Gender")
plt.xlabel("Gender")
plt.ylabel("Count")

plt.show()
```



The distribution analysis shows how demographic attributes are represented in the dataset. Understanding the balance of these groups is important because highly imbalanced distributions may cause models to learn biased patterns that favour the majority group.

10.5. Relationship Between Sensitive Features and Dropout

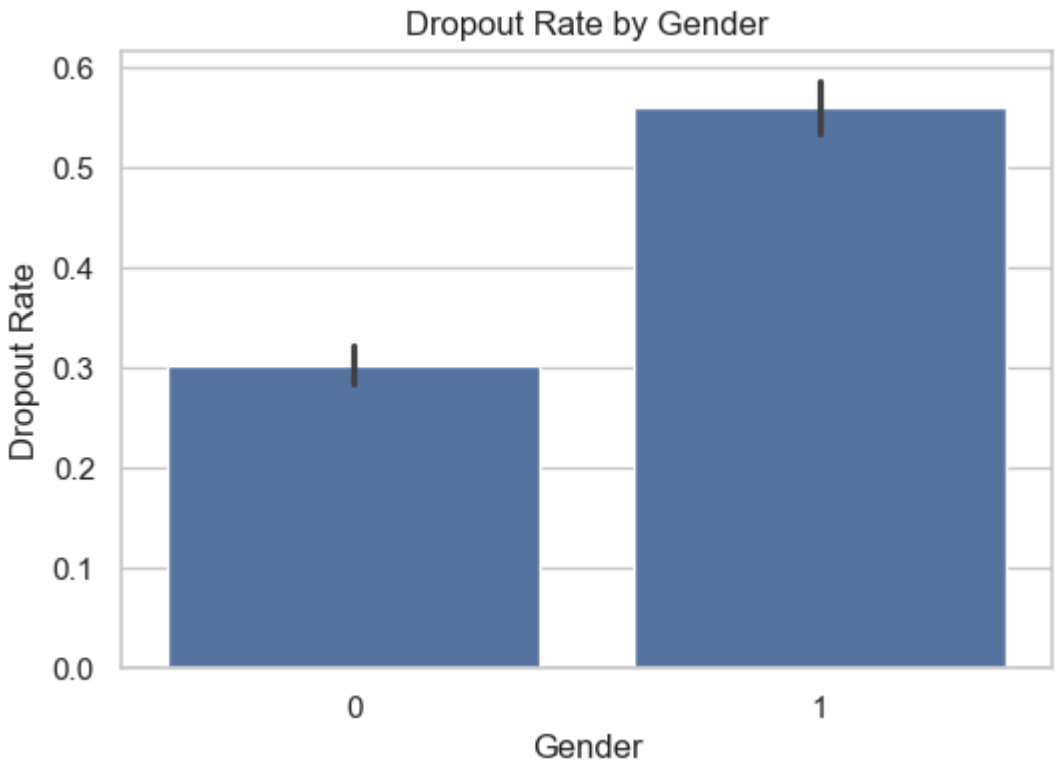
In [68]:
```
for feature in sensitive_features:
  print(f"\nDropout rate by {feature}:")
  print(df.groupby(feature)["Target"].mean())
```

```
Dropout rate by Gender:
Gender
0    0.302394
1    0.561249
Name: Target, dtype: float64

Dropout rate by Nacionality:
Nacionality
1        0.391930
6        0.500000
11       0.000000
22       0.333333
24       0.200000
26       0.111111
41       0.437500
100      1.000000
101      0.500000
103      0.333333
Other    0.400000
Name: Target, dtype: float64

Dropout rate by Mothers qualification:
Mothers qualification
1        0.346821
2        0.333333
3        0.412463
4        0.235294
5        0.500000
12       0.564103
19       0.348777
34       0.755906
37       0.455952
38       0.297240
Other    0.531250
Name: Target, dtype: float64

Dropout rate by Fathers qualification:
Fathers qualification
1        0.383880
2        0.511628
3        0.409091
4        0.437500
5        0.666667
12       0.411765
19       0.336306
34       0.743119
37       0.427723
38       0.290435
Other    0.613333
Name: Target, dtype: float64

Dropout rate by Mothers occupation:
Mothers occupation
0        0.692308
1        0.448276
2        0.425000
3        0.349265
4        0.370149
5        0.357798
6        0.337662
```

```
7        0.357143
9        0.373191
90       0.728571
Other    0.357143
Name: Target, dtype: float64

Dropout rate by Fathers occupation:
Fathers occupation
0        0.656000
2        0.476190
3        0.355140
4        0.445513
5        0.361858
6        0.355670
7        0.331532
8        0.357414
9        0.381346
10       0.400943
Other    0.461224
Name: Target, dtype: float64
```

In [69]:

```
gender_dropout = df.groupby("Gender")["Target"].mean()

plt.figure(figsize=(6,4))
sns.barplot(x="Gender", y="Target", data=df)

plt.title("Dropout Rate by Gender")
plt.ylabel("Dropout Rate")

plt.show()
```



The analysis shows that several demographic attributes display different dropout rates across groups.

In particular, `gender` shows a noticeable difference in dropout probability, where one group has a dropout rate of approximately **56%**, compared with approximately **30%** for the other group. This suggests that predictive models may learn patterns associated with gender.

It is important to note that differences in dropout rates across demographic groups do not necessarily indicate `discriminatory behaviour` in the predictive model. However, these differences suggest that demographic attributes may influence model predictions in ways that raise ethical considerations when such systems are used in decision-support contexts.

Nationality shows a highly imbalanced distribution, with the majority of observations belonging to a single category. Several other nationality groups contain very few observations, making their calculated dropout rates statistically unreliable.

Parental education and occupation variables contain many categorical levels representing different socioeconomic backgrounds. These attributes may act as proxies for family socioeconomic status and therefore introduce unintended bias into predictive models.

10.6. Proxy Variables and Indirect Bias

Even when sensitive demographic attributes are removed from a dataset, other variables may still indirectly encode similar information. These variables are often referred to as **proxy variables**.

For example, certain academic programs or socioeconomic indicators may correlate with demographic characteristics such as `gender` or `nationality`. As a result, removing sensitive attributes does not completely eliminate the possibility of bias in machine learning models.

However, excluding these attributes represents an important step toward reducing the likelihood that predictive models rely directly on demographic identity when making predictions.

10.7. Ethical Feature Selection Decision

Based on the previous analysis, several demographic and socioeconomic variables have been identified as potentially sensitive attributes. These features may influence predictive outcomes in ways that raise ethical concerns related to fairness and discrimination.

To evaluate whether similar predictive performance can be achieved without relying on demographic attributes, a second set of models will be trained without these features.

The following variables will therefore be removed from the training dataset when constructing the ethical models:

- Gender
- Nacionality

- Mothers qualification
- Fathers qualification
- Mothers occupation
- Fathers occupation

The models trained without these variables will be referred to as `ethical models`, and their performance will be compared with the original models trained on the full dataset.

11. Ethical Model Training

In this section, a second set of models is trained using a dataset that excludes the sensitive features identified during the ethical feature analysis.

The purpose of this experiment is to evaluate whether comparable predictive performance can be achieved without relying on demographic or socioeconomic attributes. Removing such variables may help reduce the risk of bias and improve the ethical suitability of the predictive system in real-world educational environments.

The following sensitive attributes are removed from the dataset:

- Gender
- Nacionality
- Mothers qualification
- Fathers qualification
- Mothers occupation
- Fathers occupation

The resulting models will be referred to as **ethical models**, and their performance will later be compared with the original models trained using the full dataset.

11.1. Ethical Dataset Preparation

To construct the ethical dataset, the sensitive demographic and socioeconomic features identified in Section 10 are removed from the feature matrix.

This ensures that the models rely only on academic, administrative, and macroeconomic indicators that are less likely to introduce demographic bias.

The resulting dataset is then used to retrain the machine learning models.

11.1.1. Define Sensitive Features

In [70]:
```
# Sensitive prefixes identified during ethical analysis
sensitive_prefixes = [
  "Gender",
  "Nacionality",
  "Mothers qualification",
  "Fathers qualification",
  "Mothers occupation",
  "Fathers occupation"
]
```

11.1.2. Create Ethical Dataset

In [71]:
```
cols_to_drop = [
  col for col in X.columns
  if any(col.startswith(prefix) for prefix in sensitive_prefixes)
]

X_eth = X.drop(columns=cols_to_drop)

print("Original number of features:", X.shape[1])
print("Ethical dataset number of features:", X_eth.shape[1])
```

```
Original number of features: 104
Ethical dataset number of features: 53
```

Six sensitive demographic and socioeconomic attributes were removed from the dataset, resulting in a reduced feature set used for training the ethical models.

11.1.3. Verify Removed Features

In [72]:
```
print("Removed sensitive features:")
for feature in sensitive_features:
  print("-", feature)
```

```
Removed sensitive features:
- Gender
- Nacionality
- Mothers qualification
- Fathers qualification
- Mothers occupation
- Fathers occupation
```

11.1.4. Check Correlation With Target

In [73]:
```
# Check strongest predictors after removing sensitive attributes
X_eth.corrwith(y).abs().sort_values(ascending=False).head(10)
```

Out[73]:
```
Curricular units 1st sem (approved)    0.554881
Curricular units 1st sem (grade)       0.519927
Tuition fees up to date                0.442138
Scholarship holder                     0.313018
Age at enrollment                      0.267229
Debtor                                 0.267207
Application mode_39                    0.255597
Course_9500                            0.208112
Course_Other                           0.207263
Curricular units 1st sem (enrolled)    0.161074
dtype: float64
```

The correlation analysis shows that the strongest predictors of student dropout remain primarily related to academic performance indicators, such as the number of approved curricular units and first semester grades. This indicates that removing demographic and socioeconomic attributes does not eliminate the primary predictive signals in the dataset. Instead, the model continues to rely mainly on academic indicators when making predictions.

11.1.5. Create Train/Test Split

In [74]:
```
X_train_eth, X_test_eth, y_train_eth, y_test_eth = train_test_split(
  X_eth,
  y,
  test_size=0.2,
  random_state=42,
  stratify=y
)

print("Training set:", X_train_eth.shape)
print("Test set:", X_test_eth.shape)
```

```
Training set: (2904, 53)
Test set: (726, 53)
```

11.1.6. Verify Class Distribution

In [75]:
```
print("Class distribution in ethical training set:")
print(y_train_eth.value_counts(normalize=True))
```

```
Class distribution in ethical training set:
Target
0    0.608471
1    0.391529
Name: proportion, dtype: float64
```

This confirms that removing sensitive features does not affect the class distribution of the dataset, ensuring that the ethical models are trained under comparable conditions to the original models.

11.1.7. Feature Scaling

Since Logistic Regression is sensitive to the scale of input features, the ethical dataset is standardized using the same preprocessing approach applied in the **baseline**

experiment.

The `StandardScaler` transforms the features so that they have a mean of 0 and a standard deviation of 1. This ensures that variables with larger numerical ranges do not dominate the learning process of the model.

In [76]:
```
# Initialize scaler
scaler_eth = StandardScaler()

# Fit scaler on training data
X_train_eth_scaled = scaler_eth.fit_transform(X_train_eth)

# Apply same transformation to test data
X_test_eth_scaled = scaler_eth.transform(X_test_eth)
```

11.2. Ethical Logistic Regression

The first ethical model implemented in this experiment is `Logistic Regression`.

This model is trained using the ethical dataset, which excludes demographic and socioeconomic attributes identified as sensitive during the ethical feature analysis.

The objective is to evaluate whether similar predictive performance can be achieved without relying on these attributes.

11.2.1. Train The Model

In [77]:
```
# Initialize the ethical logistic regression model
log_model_eth = LogisticRegression(
  max_iter=1000,
  class_weight='balanced'
)

# Train the model
log_model_eth.fit(X_train_eth_scaled, y_train_eth)
```

Out[77]:
```
▾ LogisticRegression
▸ Parameters
```

11.2.2. Cross-Validation

Cross-validation is used to evaluate how well the ethical model generalizes to unseen data. The model is trained and validated across multiple folds of the training dataset.

In [78]:
```
log_cv_eth = cross_val_score(
  log_model_eth,
  X_train_eth_scaled,
  y_train_eth,
  cv=5,
  scoring="f1"
)
```

```
print("Ethical Logistic Regression CV F1:", log_cv_eth)
print("Average CV F1:", log_cv_eth.mean())
```

```
Ethical Logistic Regression CV F1: [0.84164859 0.85042735 0.83595506 0.84651163
0.84463895]
Average CV F1: 0.8438363148415133
```

11.2.3. Make Predictions

In [79]:
```
# Predict on test set
y_pred_eth = log_model_eth.predict(X_test_eth_scaled)
```

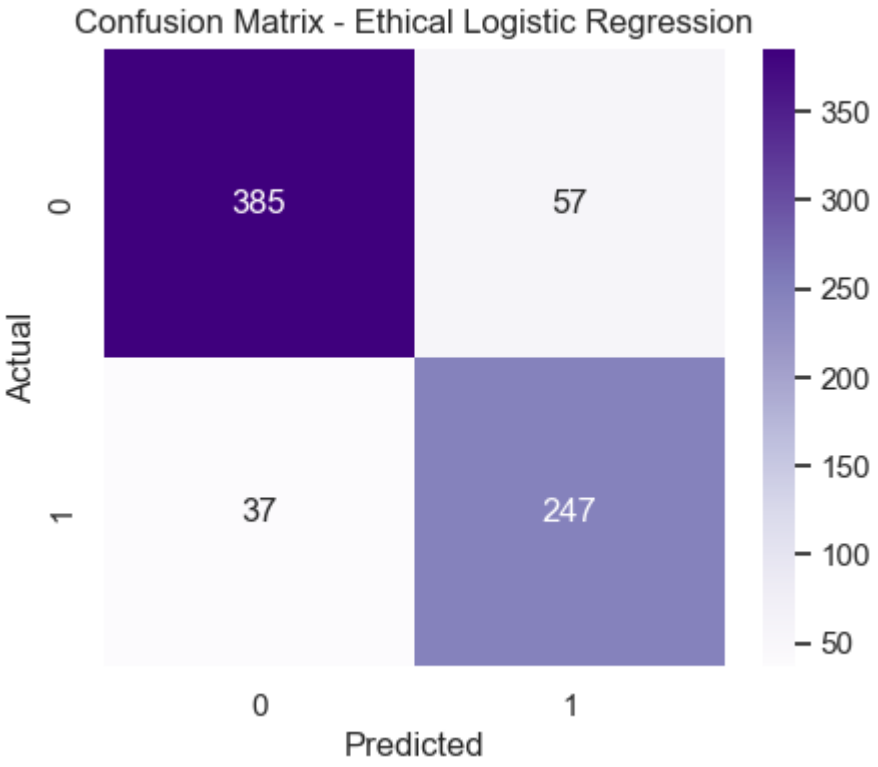
11.2.4. Evaluate the Model

In [80]:
```
eth_accuracy = accuracy_score(y_test_eth, y_pred_eth)
eth_precision = precision_score(y_test_eth, y_pred_eth)
eth_recall = recall_score(y_test_eth, y_pred_eth)
eth_f1 = f1_score(y_test_eth, y_pred_eth)

print("Accuracy:", eth_accuracy)
print("Precision:", eth_precision)
print("Recall:", eth_recall)
print("F1 Score:", eth_f1)
```

```
Accuracy: 0.8705234159779615
Precision: 0.8125
Recall: 0.8697183098591549
F1 Score: 0.8401360544217688
```

11.2.5. Confusion Matrix

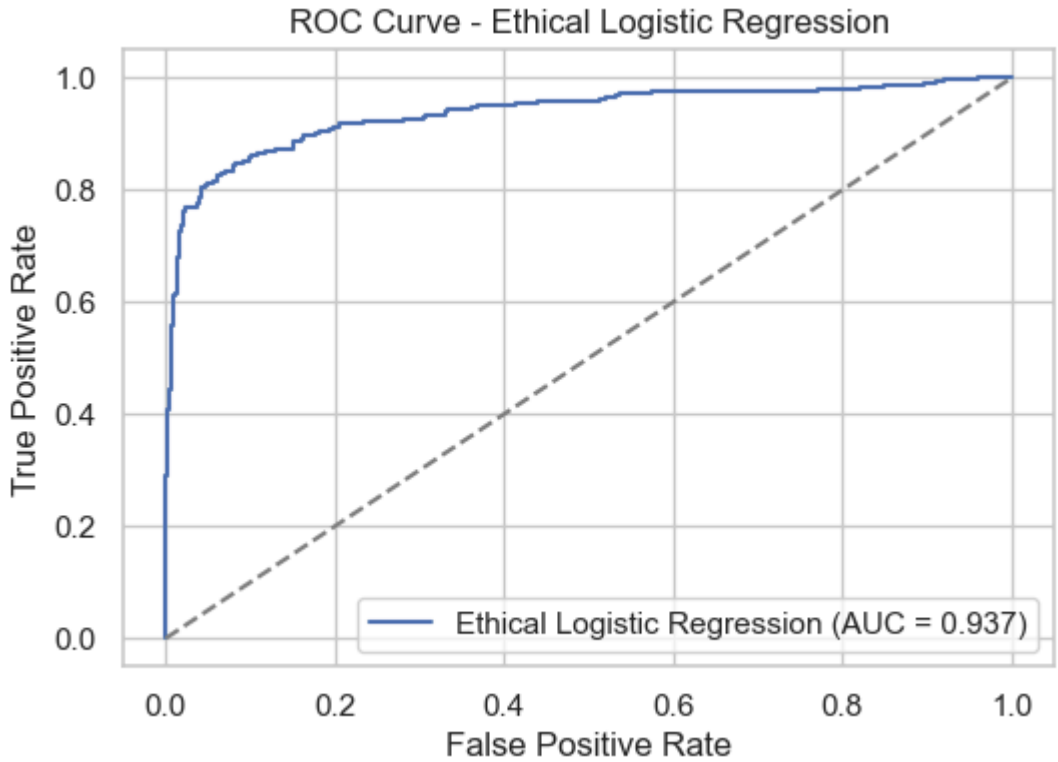
In [81]:
```
cm_eth = confusion_matrix(y_test_eth, y_pred_eth)

plt.figure(figsize=(5,4))
sns.heatmap(cm_eth, annot=True, fmt="d", cmap="Purples")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Ethical Logistic Regression")

plt.show()
```



11.2.6. ROC Curve

In [82]:
```
# Calculate predicted probabilities
y_probs_eth = log_model_eth.predict_proba(X_test_eth_scaled)[:, 1]

# Calculate ROC-AUC score
roc_auc_eth = roc_auc_score(y_test_eth, y_probs_eth)

print("ROC-AUC Score:", roc_auc_eth)

# Compute ROC curve
fpr_eth, tpr_eth, thresholds = roc_curve(y_test_eth, y_probs_eth)

plt.figure(figsize=(6,4))

plt.plot(fpr_eth, tpr_eth, label=f"Ethical Logistic Regression (AUC = {roc_auc_e
plt.plot([0,1], [0,1], linestyle="--", color="gray")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Ethical Logistic Regression")
plt.legend()

plt.show()
```

```
ROC-AUC Score: 0.9368188770632848
```



The ROC curve evaluates the classification ability of the ethical Logistic Regression model. The high ROC-AUC score indicates that the model can effectively distinguish between students who are likely to drop out and those who are likely to graduate, even after removing sensitive demographic attributes.

11.2.7. Model Performance Interpretation

The ethical Logistic Regression model achieved an accuracy of approximately **87.05%**, with a precision score of **0.81**, a recall score of **0.86**, and an F1-score of **0.84**.

The confusion matrix shows that the model correctly classified **385** students who graduated and **247** students who dropped out. However, **57** students were incorrectly predicted as dropouts, while **37** dropout cases were incorrectly classified as graduates.

Compared with the baseline Logistic Regression model trained on the full dataset, the ethical model demonstrates very similar predictive performance. The overall accuracy and F1-score remain almost unchanged, indicating that removing demographic and socioeconomic attributes does not significantly reduce the predictive capability of the model.

The recall score is `slightly lower` than in the original model, although the difference is very small, meaning that a small number of additional dropout cases are not detected. However, the difference is minimal, suggesting that the model primarily relies on academic and administrative indicators rather than demographic attributes when identifying students at risk of dropping out.

Overall, these results indicate that it is possible to construct an effective predictive model for student dropout risk without relying on sensitive demographic features, supporting the goal of developing a more ethically responsible predictive system.

11.3. Ethical Random Forest

A Random Forest classifier is trained using the ethical dataset to evaluate how removing sensitive demographic and socioeconomic attributes affects the performance of a `non-linear` ensemble model.

Random Forest models are capable of capturing complex relationships between variables and are commonly used for structured tabular datasets. By comparing the results of this model with the baseline Random Forest model trained on the full dataset, we can assess whether the removed features contributed significantly to the model's predictive performance.

11.3.1. Train the Model

In [83]:
```
# Initialize ethical Random Forest model
rf_model_eth = RandomForestClassifier(
  n_estimators=200,
  random_state=42,
  class_weight='balanced'
)

# Train the model
rf_model_eth.fit(X_train_eth, y_train_eth)
```

Out[83]:
```
▾ RandomForestClassifier
▸ Parameters
```

11.3.2. Cross-Validation

Cross-validation is used to evaluate how well the ethical Random Forest model generalizes to unseen data by training and validating the model across multiple splits of the dataset.

In [84]:
```
rf_cv_eth = cross_val_score(
  rf_model_eth,
  X_train_eth,
  y_train_eth,
  cv=5,
  scoring="f1"
)

print("Ethical Random Forest CV F1:", rf_cv_eth)
print("Average CV F1:", rf_cv_eth.mean())
```

```
Ethical Random Forest CV F1: [0.85176471 0.85382831 0.82269504 0.85377358 0.85849
057]
Average CV F1: 0.8481104397102486
```

11.3.3. Make Predictions

In [85]:
```
# Predict on test set
rf_pred_eth = rf_model_eth.predict(X_test_eth)
```

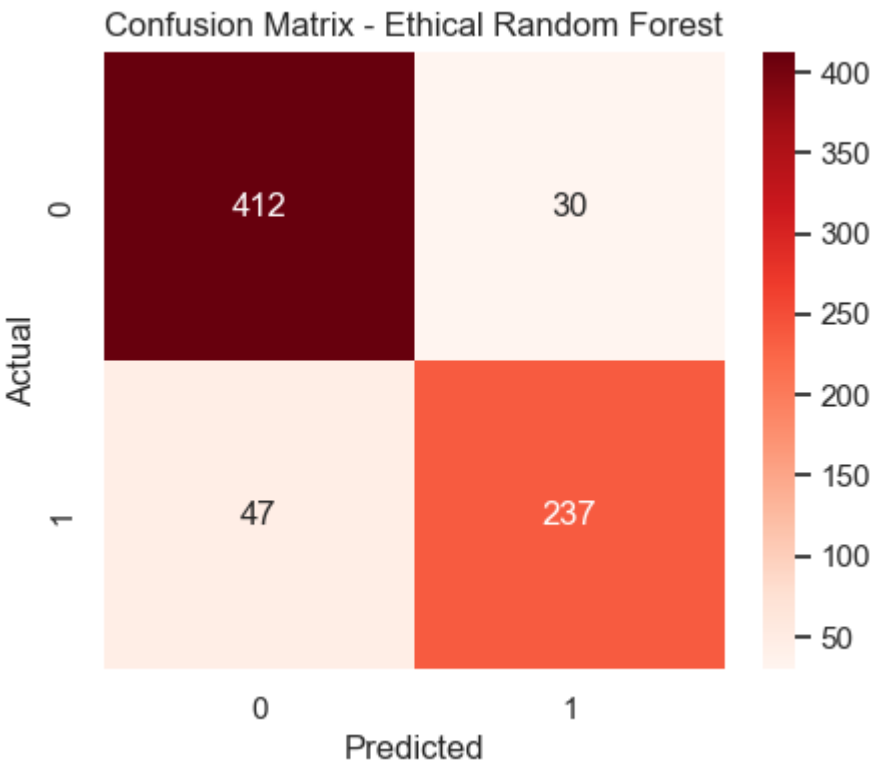
11.3.4. Evaluate the Model

In [86]:
```
rf_eth_accuracy = accuracy_score(y_test_eth, rf_pred_eth)
rf_eth_precision = precision_score(y_test_eth, rf_pred_eth)
rf_eth_recall = recall_score(y_test_eth, rf_pred_eth)
rf_eth_f1 = f1_score(y_test_eth, rf_pred_eth)

print("Accuracy:", rf_eth_accuracy)
print("Precision:", rf_eth_precision)
print("Recall:", rf_eth_recall)
print("F1 Score:", rf_eth_f1)
```

```
Accuracy: 0.8939393939393939
Precision: 0.8876404494382022
Recall: 0.8345070422535211
F1 Score: 0.8602540834845736
```

11.3.5. Confusion Matrix

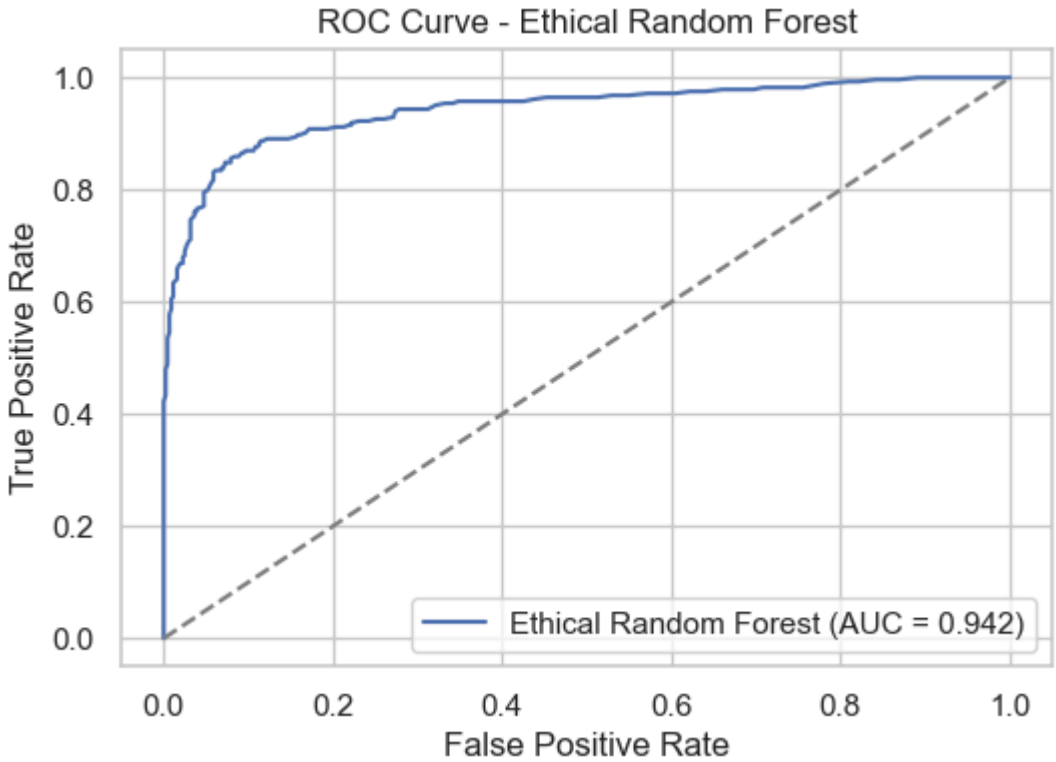
In [87]:
```
rf_cm_eth = confusion_matrix(y_test_eth, rf_pred_eth)

plt.figure(figsize=(5,4))
sns.heatmap(rf_cm_eth, annot=True, fmt="d", cmap="Reds")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Ethical Random Forest")

plt.show()
```



11.3.6. ROC Curve

In [88]:
```
# Calculate predicted probabilities
rf_probs_eth = rf_model_eth.predict_proba(X_test_eth)[:, 1]

# Calculate ROC-AUC score
rf_auc_eth = roc_auc_score(y_test_eth, rf_probs_eth)

print("ROC-AUC Score:", rf_auc_eth)

# Compute ROC curve
rf_fpr_eth, rf_tpr_eth, thresholds = roc_curve(y_test_eth, rf_probs_eth)

plt.figure(figsize=(6,4))

plt.plot(rf_fpr_eth, rf_tpr_eth, label=f"Ethical Random Forest (AUC = {rf_auc_et
plt.plot([0,1], [0,1], linestyle="--", color="gray")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Ethical Random Forest")
plt.legend()

plt.show()
```

```
ROC-AUC Score: 0.9423793894589256
```



The ROC curve demonstrates that the ethical Random Forest model maintains strong classification performance even after removing sensitive features. The ROC-AUC score indicates that the model continues to effectively separate students at risk of dropping out from those likely to graduate.

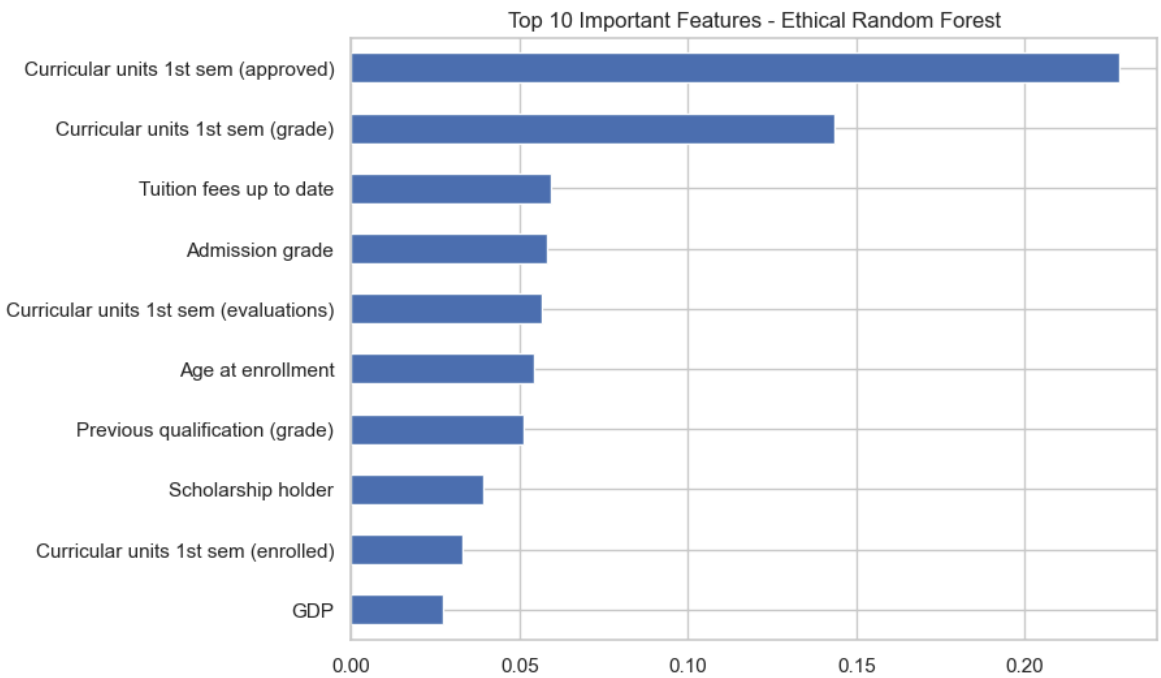
11.3.7. Feature Importance

Feature importance analysis helps identify which variables contribute the most to the predictions made by the ethical Random Forest model.

In [89]:
```
importances_eth = pd.Series(
  rf_model_eth.feature_importances_,
  index=X_eth.columns
).sort_values(ascending=False)

print(importances_eth.head(10))
```

```
Curricular units 1st sem (approved)       0.228023
Curricular units 1st sem (grade)          0.143559
Tuition fees up to date                   0.059240
Admission grade                           0.058059
Curricular units 1st sem (evaluations)    0.056605
Age at enrollment                         0.054341
Previous qualification (grade)            0.051002
Scholarship holder                        0.039240
Curricular units 1st sem (enrolled)       0.032926
GDP                                       0.027127
dtype: float64
```

In [90]:
```
plt.figure(figsize=(8,6))
importances_eth.head(10).plot(kind="barh")

plt.title("Top 10 Important Features - Ethical Random Forest")

plt.gca().invert_yaxis()

plt.show()
```



11.3.8. Model Performance Interpretation

The ethical Random Forest model achieved an accuracy of approximately **89.39%**, with a precision score of **0.88**, a recall score of **0.83**, and an F1-score of **0.86**.

The confusion matrix shows that the model correctly classified **412** students who graduated and **237** students who dropped out. However, **30** graduates were incorrectly predicted as dropouts, while **47** dropout cases were incorrectly classified as graduates.

Compared with the baseline Random Forest model trained on the full dataset, the ethical model shows a slight `reduction` in predictive performance. The accuracy, recall, and F1-score are marginally lower, indicating that removing demographic and socioeconomic attributes results in a small decrease in predictive capability.

Despite this small reduction, the overall performance of the model remains strong. Feature importance analysis reveals that the most influential predictors are primarily related to **academic performance indicators**, such as the number of approved curricular units and first semester grades.

This suggests that the model's predictive power largely depends on academic variables rather than demographic attributes. As a result, it is possible to construct an effective dropout prediction model while reducing reliance on sensitive demographic information.

11.4. Ethical Gradient Boosting

In this section, a Gradient Boosting classifier is trained using the ethical dataset that excludes sensitive demographic and socioeconomic attributes.

Gradient Boosting is an ensemble learning method that builds models sequentially, where each new model attempts to correct the errors made by previous models. This approach often achieves strong predictive performance on structured tabular datasets.

By comparing the results of this ethical Gradient Boosting model with the baseline Gradient Boosting model trained on the full dataset, we can evaluate whether removing sensitive attributes significantly affects the predictive performance of the model.

11.4.1. Train the Model

In [91]:
```
# Initialize ethical Gradient Boosting model
gb_model_eth = GradientBoostingClassifier(
  n_estimators=200,
  learning_rate=0.05,
  random_state=42
)

# Train the model
gb_model_eth.fit(X_train_eth, y_train_eth)
```

Out[91]:
```
▾ GradientBoostingClassifier
▸ Parameters
```

11.4.2. Make Predictions

In [92]:
```
# Predict on test set
gb_pred_eth = gb_model_eth.predict(X_test_eth)
```

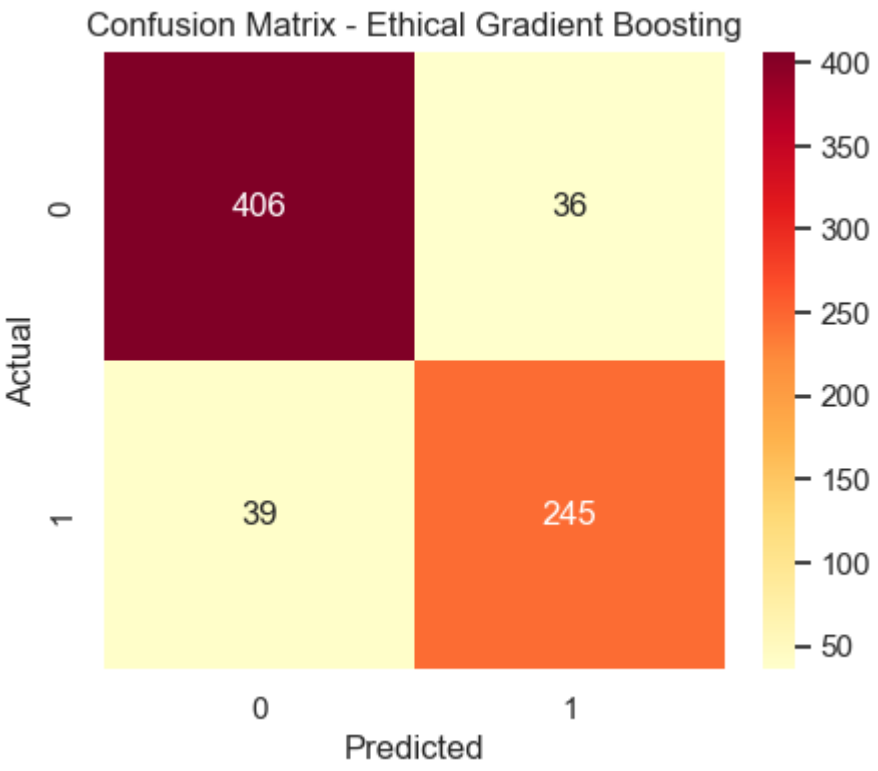
11.4.3. Evaluate the Model

In [93]:
```
gb_eth_accuracy = accuracy_score(y_test_eth, gb_pred_eth)
gb_eth_precision = precision_score(y_test_eth, gb_pred_eth)
gb_eth_recall = recall_score(y_test_eth, gb_pred_eth)
gb_eth_f1 = f1_score(y_test_eth, gb_pred_eth)

print("Accuracy:", gb_eth_accuracy)
print("Precision:", gb_eth_precision)
print("Recall:", gb_eth_recall)
print("F1 Score:", gb_eth_f1)
```

```
Accuracy: 0.8966942148760331
Precision: 0.8718861209964412
Recall: 0.8626760563380281
F1 Score: 0.8672566371681416
```

11.4.4. Confusion Matrix

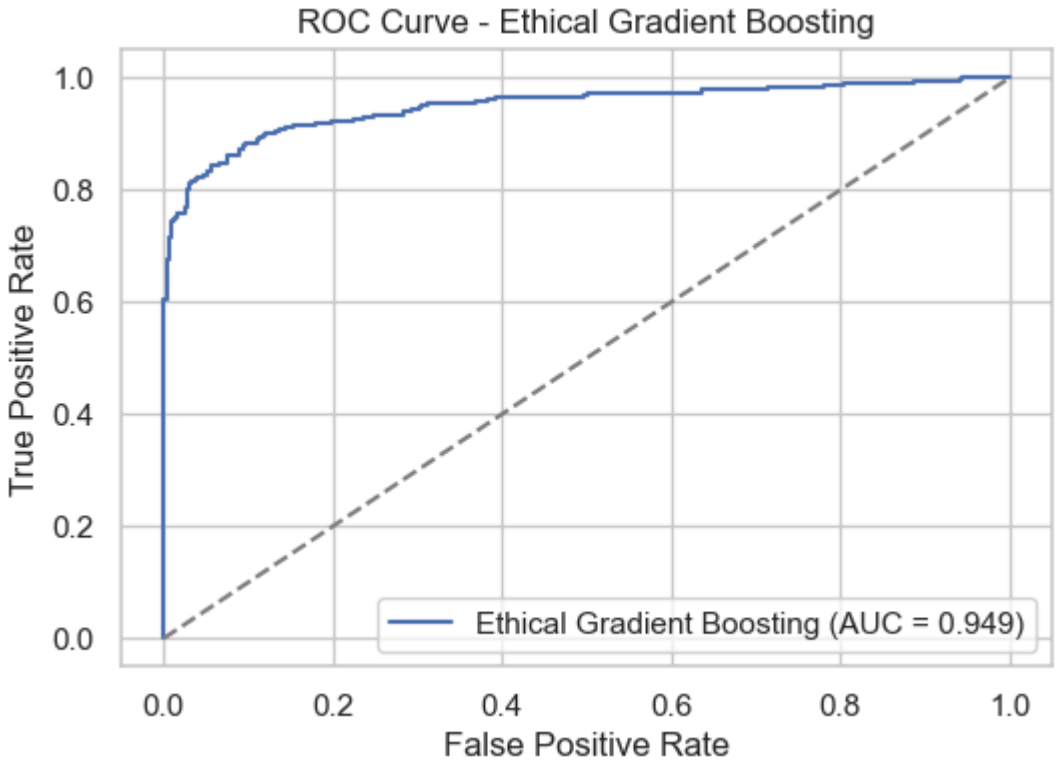
In [94]:
```
gb_cm_eth = confusion_matrix(y_test_eth, gb_pred_eth)

plt.figure(figsize=(5,4))
sns.heatmap(gb_cm_eth, annot=True, fmt="d", cmap="YlOrRd")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Ethical Gradient Boosting")

plt.show()
```



11.4.5. ROC Curve

In [95]:
```
# Calculate predicted probabilities
gb_probs_eth = gb_model_eth.predict_proba(X_test_eth)[:, 1]

# Calculate ROC-AUC score
gb_auc_eth = roc_auc_score(y_test_eth, gb_probs_eth)

print("ROC-AUC Score:", gb_auc_eth)

# Compute ROC curve
gb_fpr_eth, gb_tpr_eth, thresholds = roc_curve(y_test_eth, gb_probs_eth)

plt.figure(figsize=(6,4))

plt.plot(gb_fpr_eth, gb_tpr_eth, label=f"Ethical Gradient Boosting (AUC = {gb_au
plt.plot([0,1], [0,1], linestyle="--", color="gray")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Ethical Gradient Boosting")
plt.legend()

plt.show()
```

```
ROC-AUC Score: 0.9487524695685424
```



The ROC curve confirms that the ethical Gradient Boosting model retains strong predictive capability. The high ROC-AUC score indicates that removing sensitive demographic attributes does not significantly reduce the model's ability to distinguish between dropout and graduate outcomes.

11.4.6. Model Performance Interpretation

The ethical Gradient Boosting model achieved an accuracy of approximately **89.67%**, with a precision score of **0.87**, a recall score of **0.86**, and an F1-score of **0.86**.

The confusion matrix shows that the model correctly classified **406** students who graduated and **245** students who dropped out. However, **36** graduates were incorrectly predicted as dropouts, while **39** dropout cases were incorrectly classified as graduates.

Compared with the baseline Gradient Boosting model trained on the full dataset, the ethical model demonstrates slightly `improved` predictive performance. The accuracy, precision, recall, and F1-score are all marginally higher than those of the original model.

These results suggest that removing demographic and socioeconomic attributes does not negatively impact the model's predictive capability. Instead, the model continues to rely primarily on academic and administrative indicators, such as course performance and enrollment-related variables.

Overall, the ethical Gradient Boosting model achieves strong predictive performance while avoiding the use of potentially sensitive demographic attributes. This supports the feasibility of developing effective student dropout prediction systems that prioritize fairness and responsible data usage.

11.4.7. Threshold Tuning

11.4.7.1. Motivation

In classification models, predictions are based on probabilities. By default, a threshold of **0.5** is used to convert these probabilities into class labels. However, this threshold may not be optimal for all use cases.

In this project, the primary objective is to identify students at risk of dropping out. Therefore, `recall` is a critical metric, as failing to detect at-risk students may prevent timely intervention.

Threshold tuning is applied to the `Ethical Gradient Boosting` model, as it achieved the best overall performance in previous experiments. Adjusting the threshold allows the model to better align with the real-world objective of maximizing early detection of dropout cases.

11.4.7.2. Threshold Evaluation

In [96]:
```
thresholds = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]

threshold_results = []

for t in thresholds:
  y_pred_t = (gb_probs_eth > t).astype(int)

  precision_t = precision_score(y_test_eth, y_pred_t)
  recall_t = recall_score(y_test_eth, y_pred_t)
  f1_t = f1_score(y_test_eth, y_pred_t)
```

```
  threshold_results.append([t, precision_t, recall_t, f1_t])

threshold_df = pd.DataFrame(
  threshold_results,
  columns=["Threshold", "Precision", "Recall", "F1 Score"]
)

threshold_df

# set_index("Threshold")[["Precision", "Recall", "F1 Score"]].plot(figsize=(8,5)
# plt.title("Threshold vs Performance Metrics")
# plt.ylabel("Score")
# plt.show()
```

Out[96]:

	Threshold	Precision	Recall	F1 Score
0	0.1	0.576842	0.964789	0.722003
1	0.2	0.704787	0.933099	0.803030
2	0.3	0.796923	0.911972	0.850575
3	0.4	0.837209	0.887324	0.861538
4	0.5	0.871886	0.862676	0.867257
5	0.6	0.910853	0.827465	0.867159
6	0.7	0.945833	0.799296	0.866412
7	0.8	0.959821	0.757042	0.846457
8	0.9	0.989189	0.644366	0.780384

Different classification thresholds were evaluated to analyze how the model's performance changes across varying decision boundaries. Lower thresholds increase recall by identifying more students as at risk of dropping out, while higher thresholds improve precision by reducing false positives. This trade-off is important when selecting an appropriate threshold for real-world applications.

11.4.7.3. Selection of Optimal Threshold

Based on the evaluation results, different thresholds provide different trade-offs between precision and recall.

A threshold of **0.5** achieves the highest F1-score (0.867), indicating the best overall balance between precision and recall. However, the recall at this threshold is slightly lower compared to lower thresholds.

Lower thresholds such as **0.3** increase recall significantly (approximately 0.912), allowing the model to identify a larger proportion of students at risk of dropping out. Although precision decreases (to approximately 0.797), this trade-off is acceptable given the objective of early intervention.

Therefore, a threshold of **0.3** is selected to prioritize recall, ensuring that more at-risk students are detected by the model.

11.4.7.4. Final Model Evaluation

In [97]:
```
best_threshold = 0.3

y_pred_custom = (gb_probs_eth > best_threshold).astype(int)

print("Final Model with Custom Threshold:")

print("Accuracy:", accuracy_score(y_test_eth, y_pred_custom))
print("Precision:", precision_score(y_test_eth, y_pred_custom))
print("Recall:", recall_score(y_test_eth, y_pred_custom))
print("F1 Score:", f1_score(y_test_eth, y_pred_custom))
```

```
Final Model with Custom Threshold:
Accuracy: 0.8746556473829201
Precision: 0.796923076923077
Recall: 0.9119718309859155
F1 Score: 0.8505747126436781
```

The final model is evaluated using the selected threshold of **0.3**.

With this adjusted threshold, the model achieves an accuracy of approximately **0.875**, a precision of **0.797**, a recall of **0.912**, and an F1-score of **0.851**.

Compared to the default threshold of **0.5** (recall ≈ 0.86), the recall has increased, meaning that a larger proportion of students who are at risk of dropping out are correctly identified. This is particularly important for early intervention systems, where missing at-risk students can have significant consequences.

Although precision is lower than at higher thresholds, this indicates that more students are being flagged as at risk, including some false positives. However, this trade-off is acceptable given the objective of maximizing the detection of dropout cases.

Overall, the adjusted threshold improves the model's ability to identify at-risk students while maintaining strong overall performance.

11.4.7.5. Interpretation

The results of the threshold tuning process demonstrate the importance of aligning model behavior with the objectives of the problem.

In this case, the goal is not only to achieve high overall accuracy but to identify as many students at risk of dropping out as possible. By lowering the threshold, the model becomes more sensitive to dropout cases, increasing recall significantly.

This trade-off is appropriate in an educational context, where early identification enables timely support measures such as academic assistance or financial aid. Identifying a student as potentially at risk, even if they may not drop out, is generally less harmful than failing to identify a student who truly needs support.

Overall, threshold tuning enhances the practical applicability of the model by adapting its predictions to better serve real-world decision-making requirements.

12. Model Comparison: Original vs Ethical Models

In this section, the predictive performance of the models trained using the **full** dataset is compared with the models trained using the **ethical** dataset that excludes sensitive demographic and socioeconomic attributes.

The objective of this comparison is to evaluate whether removing sensitive features significantly affects model performance and to assess the potential trade-offs between predictive accuracy and ethical considerations.

The comparison focuses on the following evaluation metrics:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

These metrics allow us to assess both the overall predictive performance and the model's ability to correctly identify students at risk of dropping out.

12.1. Comparison Table

In [98]:
```
comparison_results = pd.DataFrame({
  "Model": [
    "Logistic Regression (Original)",
    "Logistic Regression (Ethical)",
    "Random Forest (Original)",
    "Random Forest (Ethical)",
    "Gradient Boosting (Original)",
    "Gradient Boosting (Ethical)"
  ],
  "Accuracy": [
    accuracy,
    eth_accuracy,
    rf_accuracy,
    rf_eth_accuracy,
    gb_accuracy,
    gb_eth_accuracy
  ],
  "Precision": [
    precision,
    eth_precision,
```

```
    rf_precision,
    rf_eth_precision,
    gb_precision,
    gb_eth_precision
  ],
  "Recall": [
    recall,
    eth_recall,
    rf_recall,
    rf_eth_recall,
    gb_recall,
    gb_eth_recall
  ],
  "F1 Score": [
    f1,
    eth_f1,
    rf_f1,
    rf_eth_f1,
    gb_f1,
    gb_eth_f1
  ],
  "ROC-AUC": [
    roc_auc,
    roc_auc_eth,
    rf_auc,
    rf_auc_eth,
    gb_auc,
    gb_auc_eth
  ]
})

comparison_results
```

Out[98]:

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	Logistic Regression (Original)	0.866391	0.802589	0.873239	0.836425	0.942443
1	Logistic Regression (Ethical)	0.870523	0.812500	0.869718	0.840136	0.936819
2	Random Forest (Original)	0.902204	0.895911	0.848592	0.871609	0.948055
3	Random Forest (Ethical)	0.893939	0.887640	0.834507	0.860254	0.942379
4	Gradient Boosting (Original)	0.892562	0.865248	0.859155	0.862191	0.952059
5	Gradient Boosting (Ethical)	0.896694	0.871886	0.862676	0.867257	0.948752

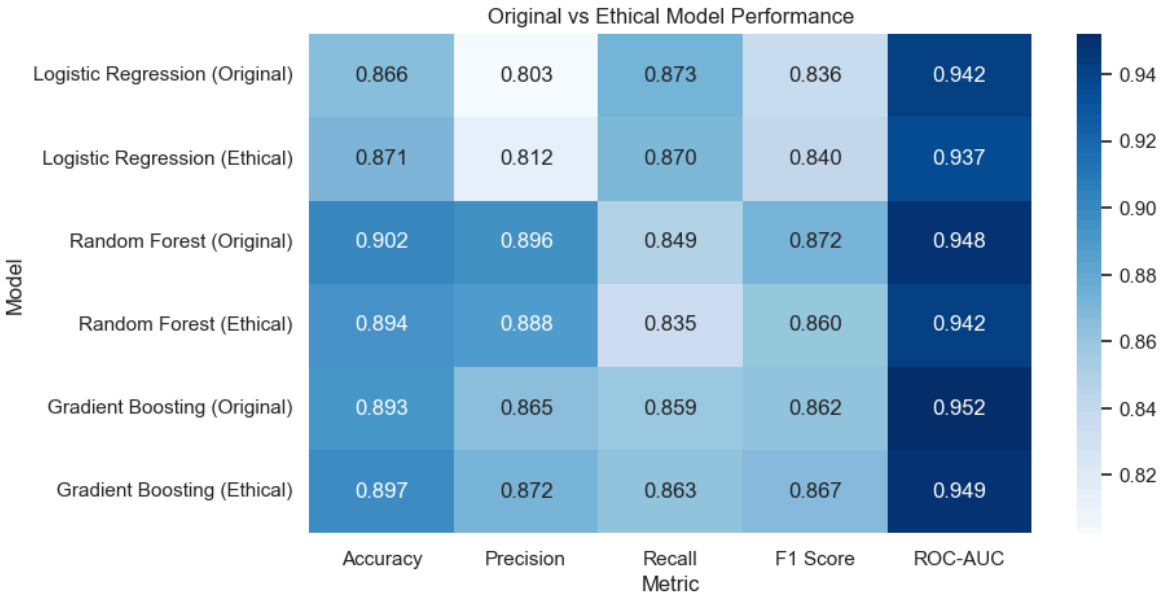
12.2. Original vs Ethical Model Performance

The following visualization compares the performance of the original models trained on the full dataset with the ethical models trained after removing sensitive demographic attributes.

In [99]:

```
plt.figure(figsize=(9, 5))

sns.heatmap(
  comparison_results.set_index("Model"),
  annot=True,
```

```
  fmt=".3f",
  cmap="Blues"
)

plt.title("Original vs Ethical Model Performance")
plt.xlabel("Metric")
plt.ylabel("Model")
plt.show()
```



The heatmap provides an overview of model performance before and after removing sensitive features. The values remain very close across the original and ethical versions of each model, suggesting that removing sensitive demographic and socioeconomic variables does not substantially reduce predictive performance.

12.3. Difference Plot

In [100…

```
difference = pd.DataFrame({
    "Model": ["Logistic Regression", "Random Forest", "Gradient Boosting"],
    "Accuracy Change": [
        eth_accuracy - accuracy,
        rf_eth_accuracy - rf_accuracy,
        gb_eth_accuracy - gb_accuracy
    ],
    "Precision Change": [
        eth_precision - precision,
        rf_eth_precision - rf_precision,
        gb_eth_precision - gb_precision
    ],
    "Recall Change": [
        eth_recall - recall,
        rf_eth_recall - rf_recall,
        gb_eth_recall - gb_recall
    ],
    "F1 Change": [
        eth_f1 - f1,
        rf_eth_f1 - rf_f1,
        gb_eth_f1 - gb_f1
    ]
```

```
})

difference.set_index("Model").plot(
    kind="bar",
    figsize=(10,6)
)

plt.axhline(0, color="black", linewidth=1)
plt.title("Performance Change After Removing Sensitive Features")
plt.ylabel("Score Difference (Ethical - Original)")
plt.xticks(rotation=0)

plt.show()
```